## 摘要

本文针对休闲零食连锁店商品销量预测问题，基于历史零售明细和天气、节假日、促销活动数据，构造日期–门店–商品粒度的日销量面板。数据清洗阶段确认销售数据覆盖 2020-04-01 至 2022-03-31，包含 7 家门店、12 个商品代码和 8 个商品类别；负销量按人工确认为损耗或冲销类负向调整，不作为正常购买需求，预测目标采用正向销量。

针对四个问题，本文采用可解释、可复现的分层建模路线。问题一使用移动平均、同星期均值和简单指数平滑分析门店、商品和门店–商品组合销量，门店层面以 7 日移动平均为主，商品和门店–商品层面以简单指数平滑为主，并补充 7 日窗口总量验证。问题二以附件类别字段作为同类零食的正式聚合依据，使用 Pearson、Spearman、去星期效应相关和门店内稳定性检查分析商品同步波动，类别预测采用“类别聚合后直接预测 + 简单指数平滑”。问题三以“合并天气 + 对数销量 + 历史控制固定效应回归”为主分析，描述性统计、温度分箱、去异常日、分门店/分类别稳健性和随机森林置换重要性作为辅助。问题四将历史销量、类别和外部变量纳入 Ridge 综合模型，并以严格 7 日递推验证作为主评价口径。

结果显示，问题一日滚动验证下门店主模型 WAPE=37.49%，商品主模型 WAPE=38.14%；问题二类别聚合后直接预测 WAPE=34.74%；问题三主回归 WAPE=84.53%。问题四严格 7 日递推验证下，问题一门店–商品简单指数平滑 WAPE=76.35%，综合 Ridge WAPE=76.68%。进一步的消融与低销量鲁棒性检验表明，低销量序列使用指数平滑、常规序列使用综合 Ridge 的混合策略 WAPE=75.85%，较两个单一模型略有改善。有限优化中近 28 日均值兜底候选在同一验证集上 WAPE 更低，但留一窗口检查未达到稳定替换阈值，因此本文仍将低销量指数平滑–常规 Ridge 混合策略作为候选最终预测方案。预测表 `outputs/final_7day_forecast_hybrid_low_volume.csv` 是在历史同期天气和活动日情景下得到的条件预测，原 `outputs/final_7day_forecast.csv` 保留为完整 Ridge 对照。



**关键词：**销量预测；移动平均；指数平滑；固定效应回归；严格递推验证；WAPE

# 1 问题重述

## 1.1 背景

休闲零食连锁门店需要根据历史销售数据安排订货、库存和促销活动。不同门店的客流基础不同，不同商品的销量规模和波动性也不同；同时，天气、节假日和门店活动可能与销售变化相关。因此，本题需要在理解历史销量结构的基础上，给出未来 7 天的销量预测，并分析商品关联和外部因素。

## 1.2 需要解决的问题

问题一要求分析不同门店和不同零食的销量情况，并分别预测未来 7 天总销量。该问题的输入是历史销售明细，输出包括门店层面、商品层面和门店–商品组合层面的预测结果。

问题二要求分析同一门店内不同零食销量之间的联系，整合同类零食，并预测未来 7 天总销量。该问题的关键是区分“商品销量同步波动”和“真实业务类别聚合”：相关性用于分析联系，附件类别字段用于正式类别预测。

问题三要求分析天气、节假日、活动日等因素与销量之间的关系。由于数据是观察数据而非随机实验，本文只讨论统计关联，不把外部变量写成严格因果影响。

问题四要求结合前三问模型和结论，预测各门店各种零食未来 7 天总销量，并与前序模型比较误差。该问题的关键是使用与未来 7 天预测一致的严格递推验证口径，避免把日滚动一步验证误写成未来 7 天一次性预测效果。

## 1.3 数据说明

附件一为历史零售明细，原始表含 67084 行、14 列，字段包括门店编号、门店名称、商品代码、商品名称、数量、销售额、收款时间和类别名称等。附件二为天气、节假日和促销活动数据，含 760 行、9 列，字段包括日期、天气、最高温、最低温、风力、节日、活动日和工休日等。字段说明见项目数据字典 DATA_DICTIONARY.md。

# 2 问题分析

本题的核心不是单一模型选择，而是按题目四问建立互相衔接的分析框架。问题一先建立可解释的销量基准模型，形成门店、商品和门店–商品三个粒度的基础预测；问题二在商品关联分析基础上进行类别聚合，解决同类零食整合预测；问题三分析外部变量与销量之间的条件统计关联，为综合模型提供特征解释；问题四在门店–商品粒度上融合历史销量、日历、类别和外部变量，并用严格 7 日递推验证检验其是否超过强基准模型。

预测评价不能随机切分，因为随机切分会让未来日期信息进入训练过程。本文使用时间顺序切分、日滚动验证和严格 7 日递推窗口验证。日滚动一步验证评价“每天更新真实销量后预测下一天”的能力；严格 7 日递推验证评价“窗口开始时一次性向未来预测 7 天”的能力，两者不能混用。

模型选择遵循三条原则：第一，所有结论来自附件数据或明确的模型计算；第二，优先使用本科数学建模竞赛中可解释、可复现、可写入论文的方法；第三，复杂模型只有在误差稳定下降且解释成本可控时才作为主方案。最终方法选择依据见 `outputs/method_search/final_method_selection.md`。

# 3 模型假设与符号说明

## 3.1 模型假设

1. 题目提供的销售明细和外部变量记录总体真实可靠。对于已发现的数据问题，本文采用记录、标记和统一口径处理，而不是直接删除原始数据。
2. 短期内历史销售规律具有一定延续性。未来 7 天预测依赖最近历史销量、同星期规律和类别/门店稳定差异。
3. 负销量不代表顾客正常购买需求。根据人工确认，负销量多对应破损、过期或冲销类负向调整，因此预测目标采用正向销量。
4. 附件类别字段可以作为同类零食的正式聚合依据。相关性聚类和销量规模分组只作为补充分析，不替代业务类别。
5. 天气、节假日和活动日与销量之间可以存在统计关联，但当前数据不足以识别严格因果效应。本文所称“影响因素分析”均指统计关联分析。
6. 未来 7 天无附件真实天气和真实活动安排。最终预测若使用天气和活动变量，应解释为“给定历史同期外部变量情景下”的条件预测。

## 3.2 符号说明

表 1: 主要符号说明

| 符号 | 含义 |
|---|---|
| $t$ | 日期序号 |
| $s$ | 门店编号 |
| $p$ | 商品编号 |
| $g$ | 商品类别 |
| $y_{s,p,t}$ | 门店 $s$、商品 $p$ 在日期 $t$ 的正向销量 |
| $\hat{y}_{s,p,t}$ | 对 $y_{s,p,t}$ 的预测值 |
| $Y_{t,g}$ | 日期 $t$ 类别 $g$ 的聚合销量 |
| $S_t$ | 指数平滑中的平滑水平 |
| $\alpha$ | 指数平滑系数 |
| $r_{pq}$ | 商品 $p$ 与商品 $q$ 的销量相关系数 |
| $X_t$ | 日历、天气、活动日等外部变量 |
| $MAE$ | 平均绝对误差 |
| $RMSE$ | 均方根误差 |
| $WAPE$ | 加权绝对百分比误差 |

# 4 数据预处理

## 4.1 数据检查

销售数据日期范围为 2020-04-01 至 2022-03-31，含 7 家门店、12 个商品代码、13 个商品名称和 8 个类别。商品代码 11001020 对应两个海苔商品名称，经人工确认以商品编号为准，名称差异视为录入笔误。天气数据日期范围为 2020-03-01 至 2022-03-30，不能覆盖销售最后一天 2022-03-31 和未来 2022-04-01 至 2022-04-07。

数据质量检查发现：销售明细中完全重复行 16605 条，经人工确认保留；负销量记录 54 条，作为损耗/冲销类负向调整；交易级数量范围为 -24 至 2712，IQR 规则标记 5536 条交易级极端记录。本文不删除原始记录，而是在处理表中保留正向销量、负向调整和异常标记。详细检查结果见 `outputs/stage1_data_inspection_report.md`。

## 4.2 日销量面板构造

本文将交易明细聚合为日期–门店–商品日销量面板，并补齐出现过的门店–商品组合在完整日期上的零销量记录。输出表 `data/processed/daily_store_product_sales.csv` 含 59130 行，日期范围为 2020-04-01 至 2022-03-31。合并外部变量后得到 `data/processed/modeling_base_table.csv`，同样为 59130 行。

日销量字段包括：

- `daily_sales`：日净销量，包含负向调整；
- `positive_sales`：正向销售数量，作为主预测目标；
- `negative_adjustment_qty`：负向调整数量绝对值；
- `has_external_data`：是否成功匹配附件二外部变量。

## 4.3 外部变量处理

附件二的“温度”和“温度.1”经人工确认为最高温和最低温，分别记为 `max_temperature` 和 `min_temperature`；“活动日”确认为门店促销活动日；“工休日”表示周末/节假日等休息日。缺少外部变量的销售日期为 2022-03-31，对应合并后 81 行 `has_external_data=0`。未来 7 天没有附件真实天气和活动日观测，本文在问题四中采用历史同期情景，并明确其条件预测性质。

# 5 问题一：门店与商品销量分析及未来 7 天预测

问题一以正向销量为目标，分别在门店、商品和门店–商品层级建模。设某一序列第 $t$ 天销量为 $y_t$。

## 5.1 模型建立

7 日移动平均模型为

$$\hat{y}_{t+1} = \frac{1}{7}\sum_{i=0}^{6} y_{t-i}. \tag{1}$$

该模型用最近 7 天平均销量代表短期水平，适合作为门店层面的主模型。

同星期均值模型为

$$\hat{y}_t = \frac{1}{m}\sum_{j=1}^{m} y_{t-7j}, \quad m \le 8. \tag{2}$$

该模型用历史相同星期几销量预测目标日，作为检验星期效应的基准模型。

简单指数平滑模型为

$$S_t = \alpha y_t + (1-\alpha)S_{t-1}, \quad \hat{y}_{t+1} = S_t, \quad 0 < \alpha \le 1. \tag{3}$$

该模型更重视近期销量，适合作为商品和门店–商品层面的主模型 [4]。

## 5.2 验证结果

阶段 2 日滚动验证结果见表 2。

表 2: 问题一日滚动验证下主模型误差

| 层级 | 主模型 | MAE | RMSE | WAPE |
|---|---|---|---|---|
| 门店 | 移动平均 (7 日) | 9.685 | 15.445 | 37.49% |
| 商品 | 简单指数平滑 | 5.748 | 10.968 | 38.14% |
| 门店–商品 | 简单指数平滑 | 1.783 | 3.733 | 74.95% |

为了使评价更贴近“未来 7 天总销量”，方法复核中补充 7 日窗口总量验证。结果显示，原问题一主方案在商品层面 WAPE=21.51%，门店层面 WAPE=22.12%，门店–商品层面 WAPE=43.06%。更复杂的滞后 Ridge 和随机森林未形成稳定优势，因此问题一保留原主模型。

## 5.3 描述性图表

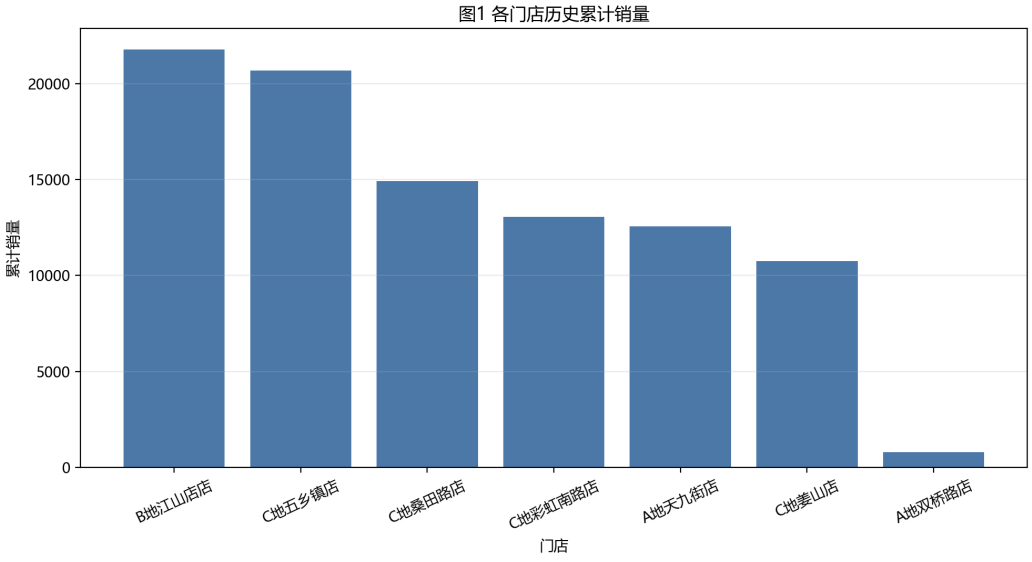



图 1: 各门店累计销量，用于说明门店销量规模差异

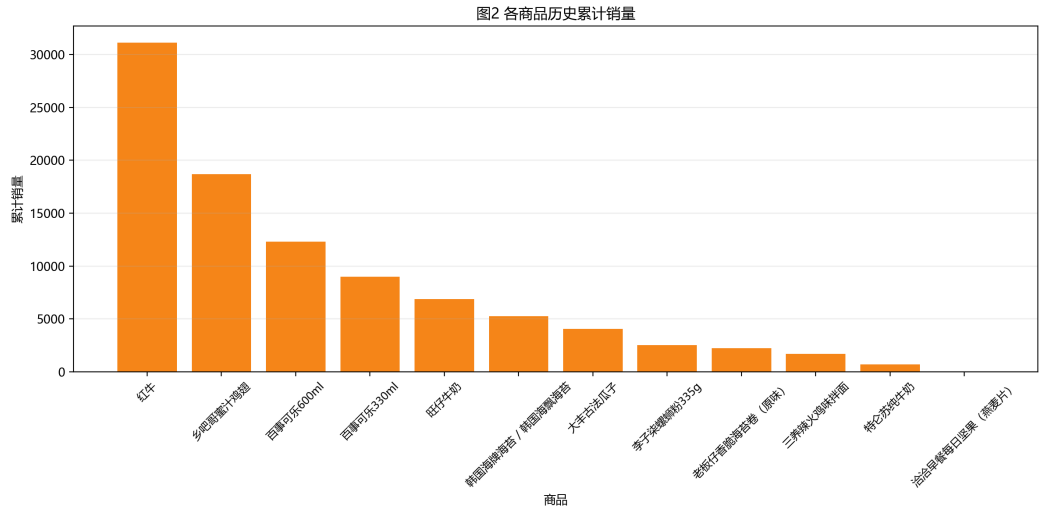



图 2: 各商品累计销量，用于说明商品销量规模差异

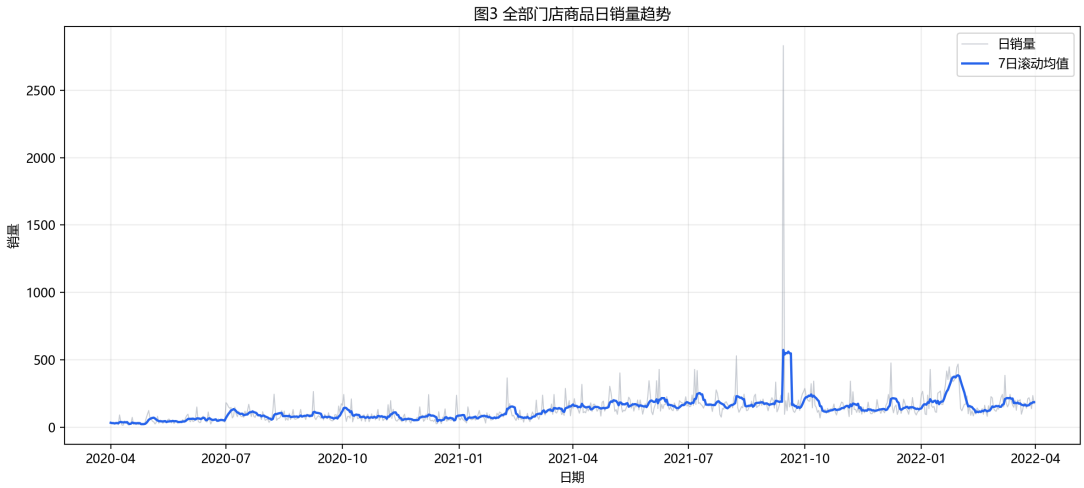



图 3: 总体日销量趋势，用于观察历史销量波动

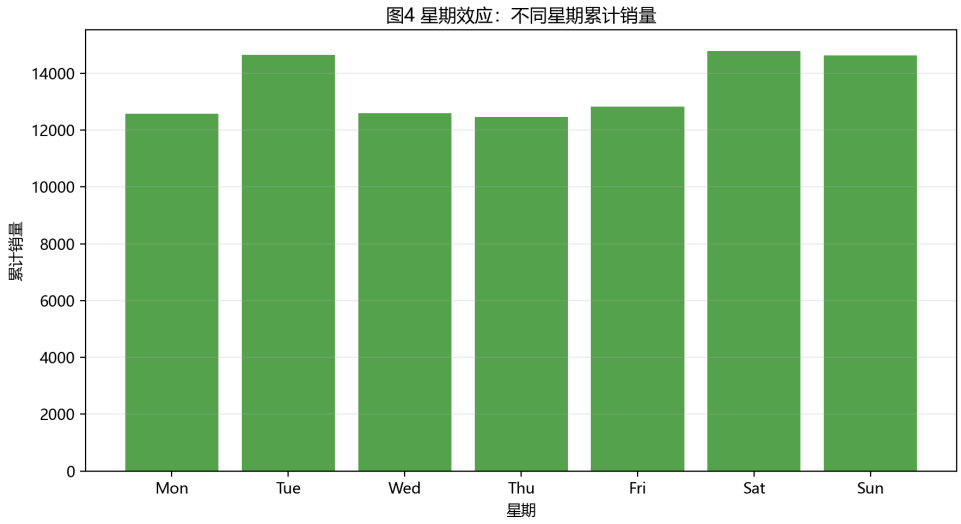



图 4: 星期效应，用于判断同星期均值模型是否有必要

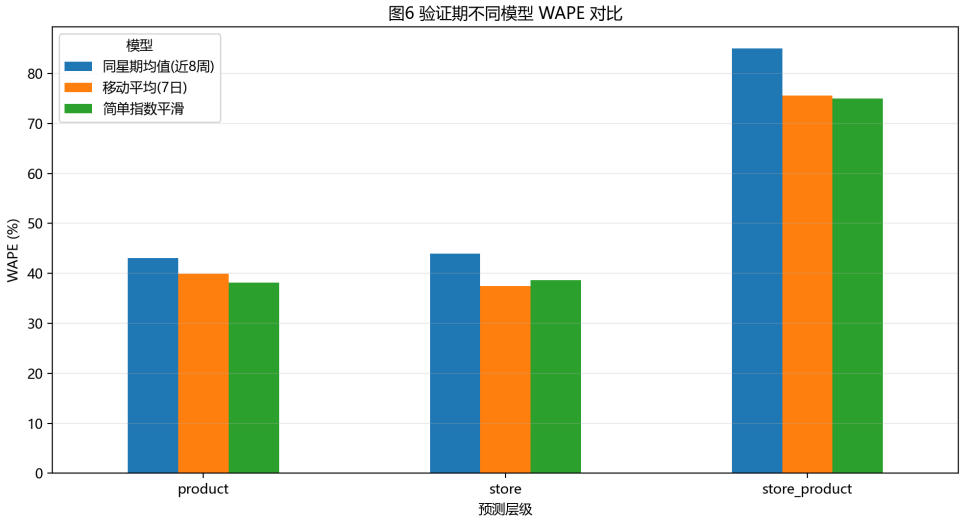



图 5: 问题一模型 WAPE 对比，用于选择各层级主模型

## 5.4 问题一结果

历史累计销量最高的门店为 B 地江山店店，累计销量 21773；历史累计销量最高的商品为红牛，累计销量 31119。门店和商品销量规模差异明显，说明不能只用统一平均水平预测所有对象。

未来 7 天门店预测中，B 地江山店店预测总销量最高，为 309.0；C 地五乡镇店为 280.0；A 地天九街店为 245.0。商品预测中，乡吧哥蜜汁鸡翅预测 389.613，红牛预测 286.719，百事可乐 600ml 预测 136.219。问题一门店层面和商品层面分别建模，预测总量没有强制一致：门店层面合计 1296.000，商品层面合计 1226.006。

# 6 问题二：商品关联分析与类别聚合预测

问题二关注同一门店中不同零食销量之间的联系，并要求整合同类零食预测未来 7 天总销量。设 $y_{t,p}$ 表示第 $t$ 天商品 $p$ 的正向销量，$c(p)$ 表示商品所属类别。

## 6.1 商品销量矩阵与相关性

对每个门店构造日期 × 商品销量矩阵：

$$Y_s = (y_{t,p}), \quad t = 1, \ldots, n, \ p = 1, \ldots, m. \tag{4}$$

商品 $p$ 与商品 $q$ 的 Pearson 相关系数为

$$r_{pq} = \frac{\sum_t (y_{t,p} - \bar{y}_p)(y_{t,q} - \bar{y}_q)}{\sqrt{\sum_t (y_{t,p} - \bar{y}_p)^2} \sqrt{\sum_t (y_{t,q} - \bar{y}_q)^2}}. \tag{5}$$

为避免单一 Pearson 相关造成过度解释，本文补充 Spearman 相关、去星期效应 Pearson 和门店内相关稳定性检查。商品关联结论只解释为同步波动，不写成因果或替代关系。

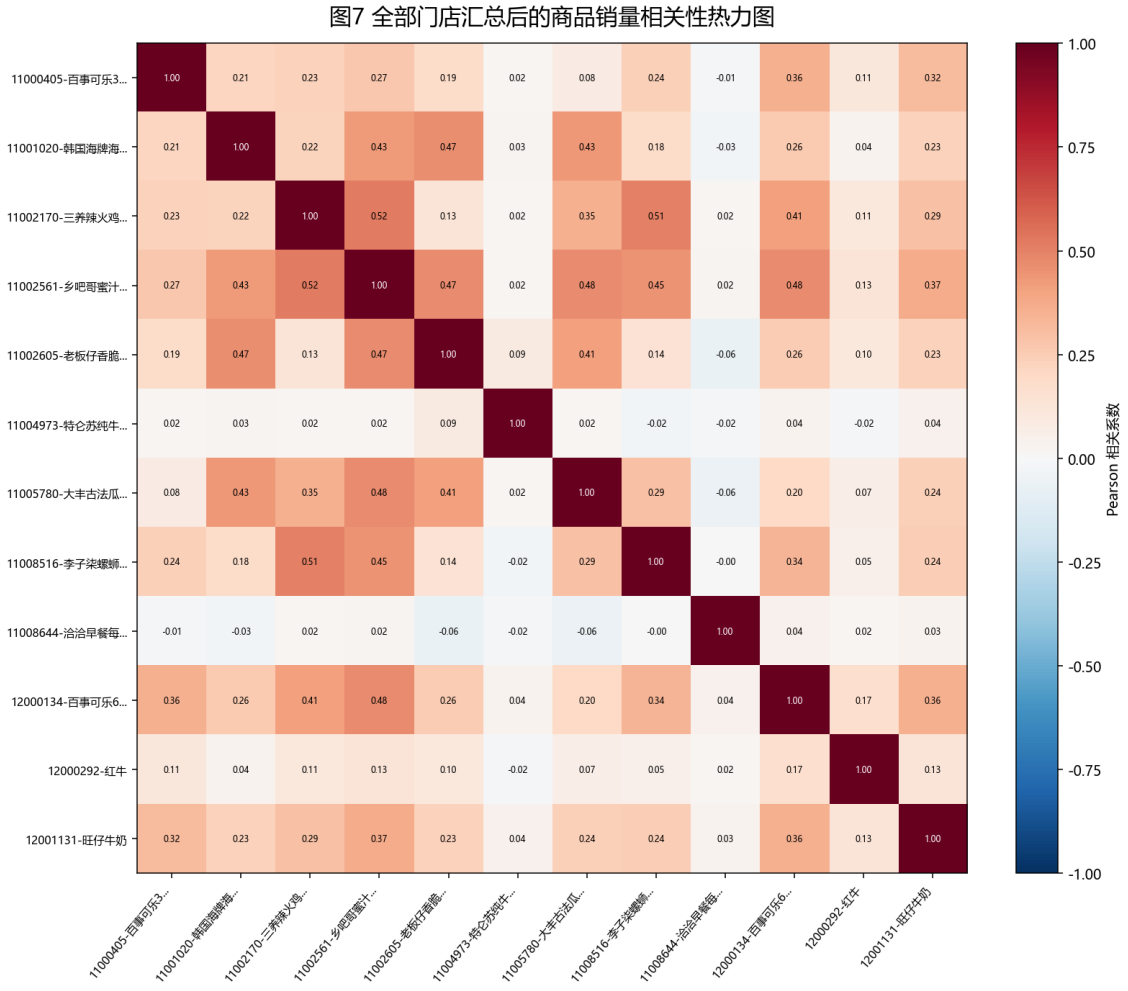



图 6: 商品相关性热力图，用于展示商品日销量同步波动结构

## 6.2 同类零食整合

类别聚合按附件 `category` 字段进行：

$$Y_{t,g} = \sum_{p:c(p)=g} y_{t,p}. \tag{6}$$

该字段具有业务含义，因此作为“同类零食”的正式聚合依据。相关性聚类和销量规模分组可作为补充探索，但不替代附件业务类别。

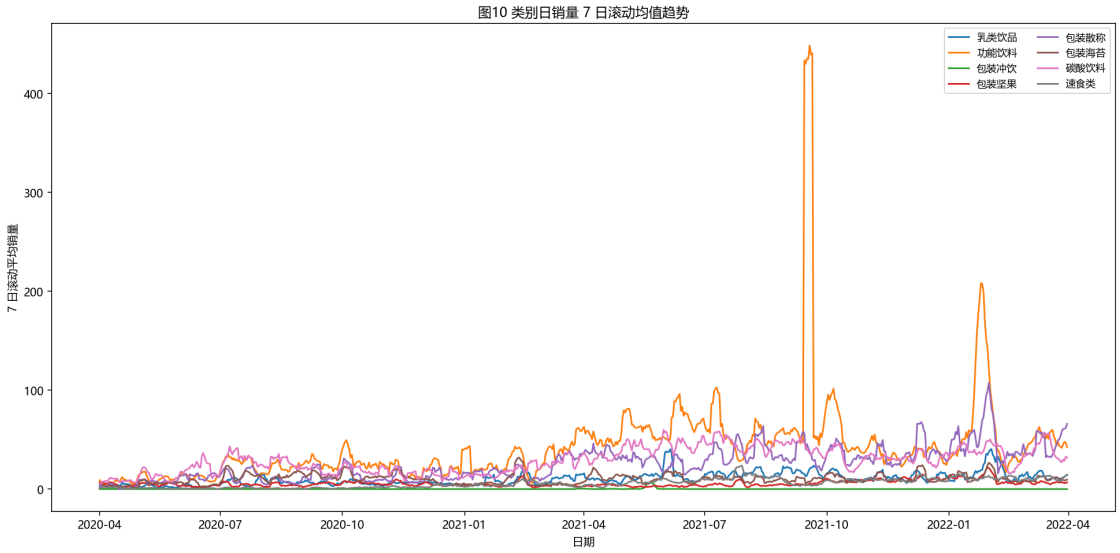



图 7: 类别日销量趋势，用于观察类别层面的时间波动

## 6.3 类别预测与误差比较

本文比较“单品预测后按类别加总”和“类别聚合后直接预测”。验证结果见表 3。

表 3: 问题二类别预测策略误差比较

| 方法 | 模型 | MAE | RMSE | WAPE |
|---|---|---|---|---|
| 类别聚合后直接预测 | 简单指数平滑 | 7.853 | 13.933 | 34.74% |
| 单品预测后按类别加总 | 简单指数平滑 | 7.873 | 13.949 | 34.83% |
| 类别聚合后直接预测 | 移动平均 (7 日) | 8.246 | 14.174 | 36.48% |
| 类别聚合后直接预测 | 同星期均值 (近 8 周) | 8.851 | 15.057 | 39.15% |

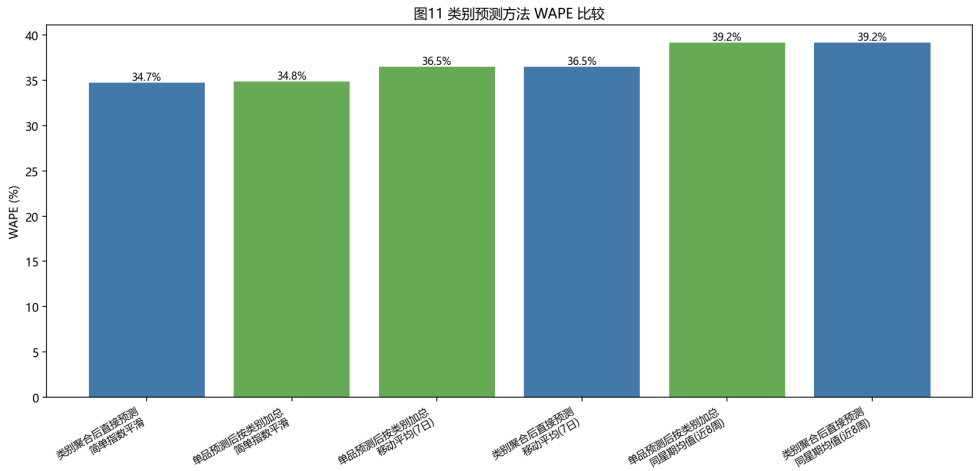



图 8: 类别预测方法 WAPE 对比，用于比较聚合后直接预测和单品预测后加总

因此，问题二主方案采用“附件类别聚合后直接预测 + 简单指数平滑”。相关性聚类 WAPE=31.02%，销量规模分组 WAPE=24.24%，但前者业务类别含义弱，后者预测对象变成高/中/低销量组，均不替代附件类别主方案。

### 6.4 问题二结果

商品关联分析表明，全部门店汇总层面存在 2 对强正相关商品对和 25 对弱相关商品对。强正相关只能说明历史日销量同步波动，可能共同受客流、消费场景或促销节奏影响，不能证明商品之间存在直接带动关系。负相关候选的相关系数绝对值较小，当前数据未发现强替代关系证据。

类别预测结果显示，未来 7 天预测最高的类别为包装散称，预测 389.613；其次为功能饮料，预测 286.719；碳酸饮料预测 247.326。类别聚合可以降低单品零销量和偶然大单带来的波动，但会损失同一类别内部商品差异。

## 7 问题三：外部因素统计关联分析

问题三分析天气、节假日、活动日等外部因素与零食销量之间的统计关联。本文将外部因素构造为分类变量和连续变量：天气类型合并为更稳定的天气组，最高温、最低温、平均温度、昼夜温差和风力为连续变量，节假日、周末、活动日为 0–1 变量，星期、月份、门店、商品和历史销量作为控制变量。原始附件中没有湿度字段，因此不构造湿度变量。

## 7.1 描述性分析

对某一外部因素 $G$，先计算不同分组下的平均销量：

$$\bar{y}_g = \frac{1}{n_g} \sum_{i:G_i=g} y_i. \tag{7}$$

该方法直观，但不能控制门店、商品和星期等混杂因素，只作为现象展示。

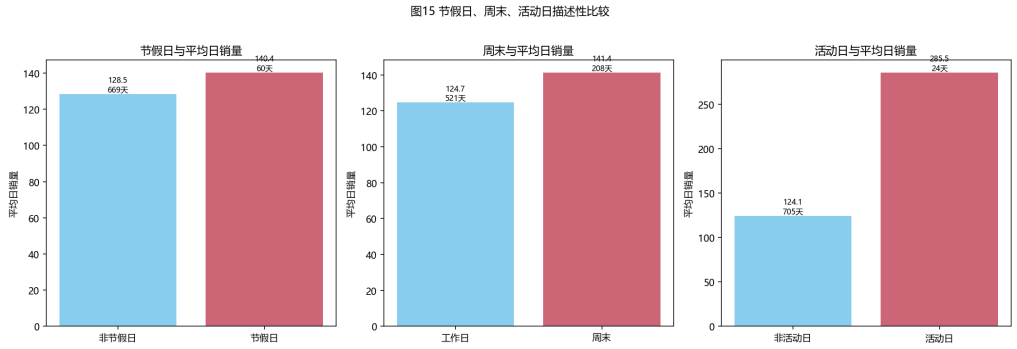



图 9: 二元外部因素均值差异，用于展示节假日、周末和活动日下的描述性销量差异

## 7.2 固定效应回归模型

问题三主模型采用“合并天气 + 对数销量 + 历史控制固定效应回归”。设 $z_{s,p,t} = \log(1 + y_{s,p,t})$，模型形式为

$$z_{s,p,t} = \beta_0 + \beta^\top External_t + \rho_1 lag7_{s,p,t} + \rho_2 rolling28_{s,p,t} + \gamma_s + \delta_p + \eta_w + \mu_m + \varepsilon_{s,p,t}. \tag{8}$$

其中，$External_t$ 包括合并天气类别、平均温度、昼夜温差、风力、节假日和活动日；$\gamma_s$、$\delta_p$、$\eta_w$、$\mu_m$ 分别表示门店、商品、星期和月份固定效应；`lag_7` 和 `rolling_28_prev` 只使用过去销量。线性回归和固定效应思想参考 [5]。

选择该模型的原因是：详细天气类别中存在雨夹雪、暴雨等少数样本天气，直接排序容易不稳；原始销量分布偏斜，对极端销量敏感；加入历史控制项后能减少基础需求差异对外部变量系数的干扰。

方法探索验证指标见表 4。

表 4: 问题三外部因素分析方法验证指标

| 方法 | MAE | RMSE | WAPE | 论文定位 |
|---|---|---|---|---|
| 原方案详细天气 FE 回归 | 1.969 | 4.144 | 88.09% | 补充 |
| 合并天气 + 对数销量 + 历史控制 FE 回归 | 1.890 | 4.448 | 84.53% | 主分析 |
| 温度分箱 + 对数销量 + 历史控制 FE 回归 | 1.889 | 4.445 | 84.51% | 非线性检查 |
| 去异常日 + 对数销量 + 历史控制 FE 回归 | 1.825 | 4.254 | 84.94% | 稳健性检查 |
| 随机森林 grouped weather + history | 1.892 | 4.178 | 84.66% | 预测贡献补充 |

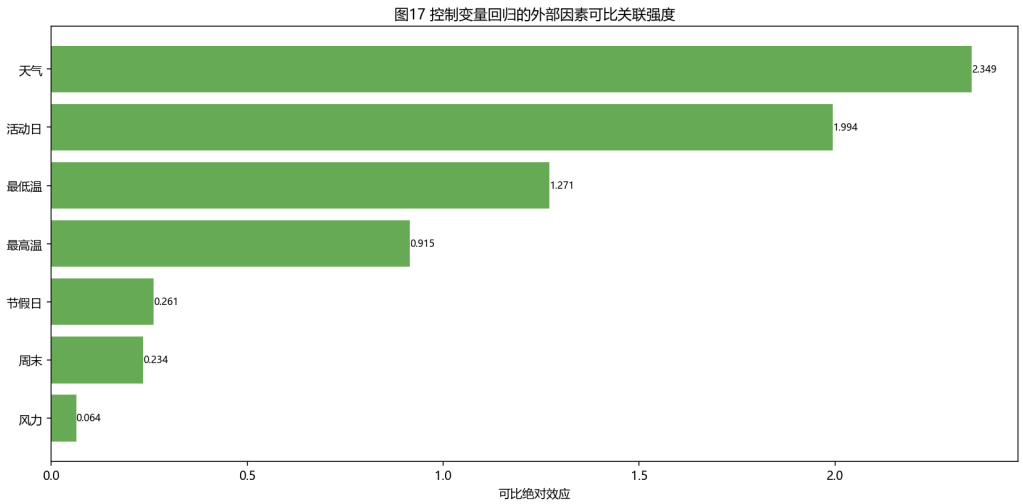



图 10: 回归关联强度排序，用于展示外部变量在控制部分混杂因素后的可比统计关联

## 7.3 机器学习辅助分析

随机森林置换重要性用于补充判断非线性预测贡献 [1]。它能说明打乱某一特征后预测误差增加多少，但不能说明销量变化方向，也不能证明因果关系。因此，问题三最终结论以回归的统计关联方向为主，以随机森林重要性作为辅助排序依据。

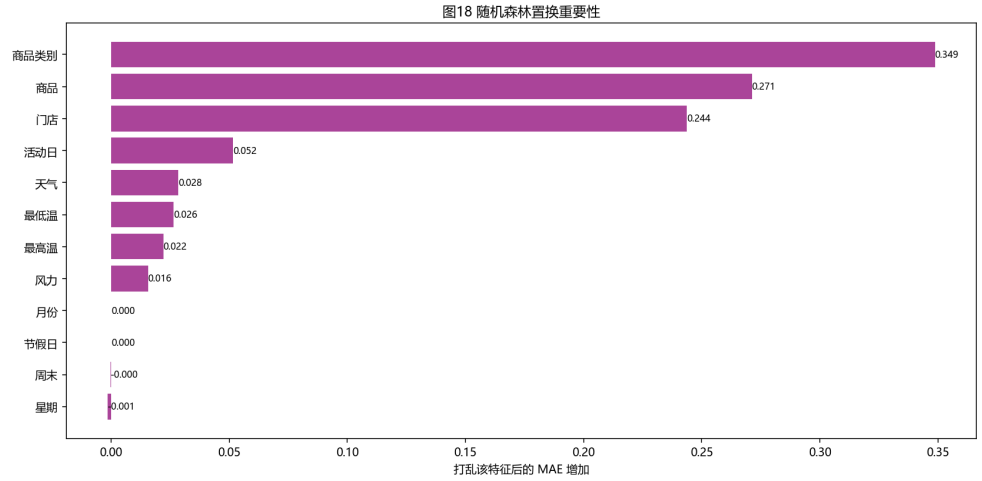



图 11: 随机森林置换重要性，用于展示非线性模型中的预测贡献

## 7.4 问题三结果

描述性统计显示，不同天气、节假日、周末和活动日下的平均销量存在差异，但这些差异可能与门店、商品、星期和月份同时相关。固定效应回归和稳健性检查表明，活动日变量与销量上升之间的统计关联相对更稳定；天气变量存在一定关联，但细分天气类别受样本量限制；节假日和周末与星期效应、月份季节性重叠，结论应作为辅助。本文不分析湿度，因为附件中没有湿度字段。

# 8 问题四：综合预测模型与严格递推验证

问题四需要在门店–商品粒度上进行综合预测。设 $y_{s,p,t}$ 表示门店 $s$、商品 $p$ 在日期 $t$ 的正向销量，综合模型的特征包括历史销量滞后项、滚动均值、日历变量、门店、商品、类别和外部变量。

## 8.1 综合模型构造

综合 Ridge 的特征包括滞后销量、滚动均值、星期、月份、是否周末、门店、商品、类别、天气、节假日和活动日。设 $x_{s,p,t}$ 为上述变量经数值化和独热编码后的特征向量，Ridge 回归估计式为

$$\hat{y}_{s,p,t} = x_{s,p,t}^{\top}\hat{\beta}, \tag{9}$$

其中参数由如下正则化最小二乘问题确定：

$$\hat{\beta} = \arg\min_{\beta} \left\{ \sum_{(s,p,t)\in\mathcal{T}} \left(y_{s,p,t} - x_{s,p,t}^{\top}\beta\right)^2 + \lambda \sum_{j=1}^{d} \beta_j^2 \right\}. \tag{10}$$

其中，$\mathcal{T}$ 为训练样本集合，$\lambda$ 为正则化强度，$d$ 为特征维数。$Lag$ 包括 `lag_1`、`lag_7`、`lag_14`，$Roll$ 包括 `rolling_mean_7` 和 `rolling_mean_14`，$Cal$ 包括星期、月份和周末标记，$External$ 包括天气、温度、风力、节假日和活动日。完整字段含义见 `outputs/q4_feature_dictionary.csv`。Ridge 回归通过 $L_2$ 正则化缓解多重共线性，适合在保留线性可解释性的同时融合多类特征 [3]。随机森林和其他树模型只作为补充比较，不作为论文主模型。

## 8.2 严格 7 日递推验证

信息泄露专项检查 `outputs/q4_information_leakage_review.md` 指出：原日滚动验证没有当天销量直接进入当天特征，但验证窗口后续日期的 `lag_1` 和滚动均值会使用窗口内前几天真实销量。因此，问题四以 `outputs/q4_recursive_7day_model_comparison.md` 中的严格 7 日递推验证作为主口径。

严格递推验证窗口见表 5。每个窗口只使用窗口起点以前的真实历史销量训练模型；窗口内第 2 至第 7 天若需要滞后或滚动特征，则使用前面已经预测出的销量，而不使用窗口内真实销量。

表 5: 问题四严格 7 日递推验证窗口

| 窗口起点 | 窗口终点 | 长度 |
|---|---|---|
| 2022-03-01 | 2022-03-07 | 7 |
| 2022-03-08 | 2022-03-14 | 7 |
| 2022-03-15 | 2022-03-21 | 7 |
| 2022-03-22 | 2022-03-28 | 7 |

严格递推下，门店–商品层级误差见表 6。

表 6: 问题四严格 7 日递推主粒度误差比较

| 模型 | MAE | RMSE | WAPE |
|---|---|---|---|
| 问题一门店–商品简单指数平滑 | 1.816 | 3.772 | 76.35% |
| 综合 Ridge 回归 | 1.824 | 3.702 | 76.68% |
| 14 日滚动均值基准模型 | 1.847 | 3.819 | 77.66% |
| 7 日移动平均基准模型 | 1.865 | 3.797 | 78.43% |
| 综合随机森林 | 1.988 | 4.842 | 83.60% |

在严格 7 日递推验证口径下，综合 Ridge 模型相较部分基准模型仍具有竞争力，但未超过问题一简单指数平滑模型。其主要价值在于统一融合门店、商品、类别、历史销量和外部因素，为多因素解释和情景预测提供一致框架。本文不作全局最优或显著提升的夸大表述。

## 8.3 消融检验与低销量混合策略

为进一步说明综合模型中不同特征的贡献，本文在相同严格 7 日递推窗口下进行消融检验，结果见表 7。消融结果表明，去除天气变量后 WAPE 从 76.68% 上升至 77.73%，去除活动日后上升至 77.22%，不使用销量历史时上升至 84.49%。这说明销量滞后和滚动统计仍是最核心的信息，天气和活动日具有一定预测补充作用，但不能解释为因果影响。

表 7: 问题四严格递推消融检验

| 模型 | MAE | RMSE | WAPE |
|---|---|---|---|
| 问题一门店–商品简单指数平滑 | 1.816 | 3.772 | 76.35% |
| 完整综合 Ridge | 1.824 | 3.702 | 76.68% |
| 去节假日活动 Ridge | 1.830 | 3.748 | 76.96% |
| 去活动日 Ridge | 1.837 | 3.748 | 77.22% |
| 去天气 Ridge | 1.849 | 3.698 | 77.73% |
| 历史日历 Ridge | 1.859 | 3.739 | 78.19% |
| 无销量历史 Ridge | 2.009 | 4.252 | 84.49% |

同时，低销量诊断显示大量门店–商品序列存在零销量和间歇需求。本文不直接引入复杂 Croston/TSB 模型，而采用一条可解释的规则化混合策略：在预测窗口开始日前，若某门店–商品组合被判为低销量序列，则使用问题一简单指数平滑；其余常规序列使用完整综合 Ridge。低销量判定只使用窗口开始日前历史数据：若正销量天数占比不高于 0.15 或历史日均销量低于 0.20，则记为 `very_sparse`；若零销量比例不低于 0.75 或历史日均销量低于 0.50，则记为 `low_volume_intermittent`；无正销量历史序列单独记为 `no_positive_history`。上述三类进入低销量兜底，其余序列仍由 Ridge 预测。该策略的严格递推验证结果见表 8。

表 8: 低销量鲁棒性策略比较

| 模型 | MAE | RMSE | WAPE |
|---|---|---|---|
| 低销量指数平滑–常规 Ridge 混合 | 1.804 | 3.718 | 75.85% |
| 问题一简单指数平滑 | 1.816 | 3.772 | 76.35% |
| 低销量收缩–常规 Ridge 混合 | 1.823 | 3.709 | 76.66% |
| 完整综合 Ridge | 1.824 | 3.702 | 76.68% |
| 低销量收缩指数平滑 | 1.835 | 3.764 | 77.15% |

该混合策略相对问题一简单指数平滑降低 0.49 个百分点，相对完整 Ridge 降低 0.83 个百分点。由于改进幅度不大且验证窗口有限，本文将其表述为低销量场景下的稳健改进，而不夸大为显著突破。完整结果见 `outputs/q4_ablation_report.md` 和 `outputs/q4_low_volume_strategy_report.md`。

本轮进一步尝试了低销量阈值调整、近 28 日均值、商品均值、类别均值和层级收缩等兜底方法，并检查门店、商品和类别层级比例校准。全样本严格递推下，“低销量

用近 28 日均值”的探索性候选 WAPE 可降至 74.95%，但留一窗口选模后的平均改善仅 0.19 个百分点，未达到 0.3 个百分点的稳定替换阈值。因此，本文不把该探索候选替换为主方案，只将其作为附加稳健性检查。详细结果见 `outputs/q4_candidate_model_review.md`。

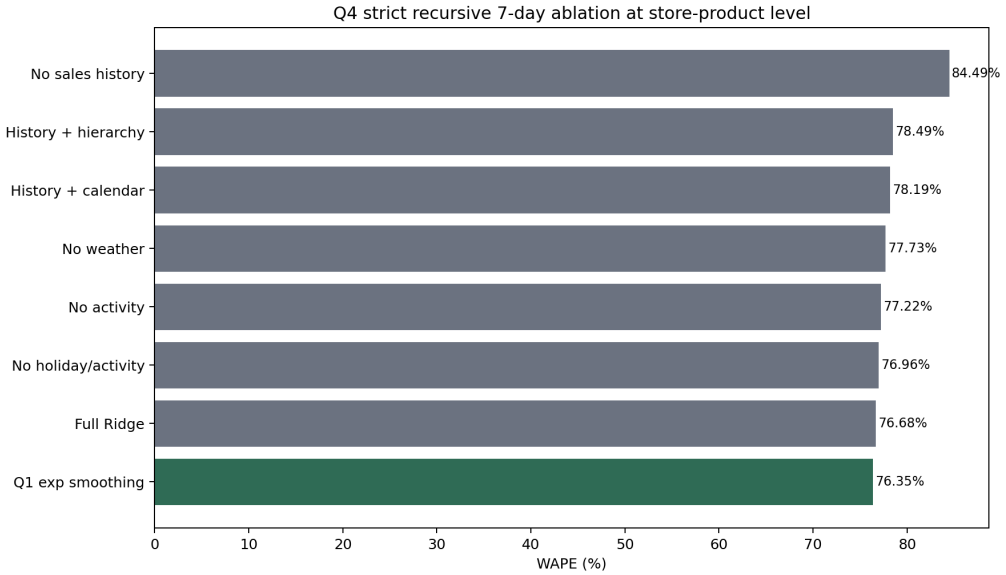



图 12: 严格递推消融检验 WAPE 对比，用于说明历史销量和外部变量的边际贡献

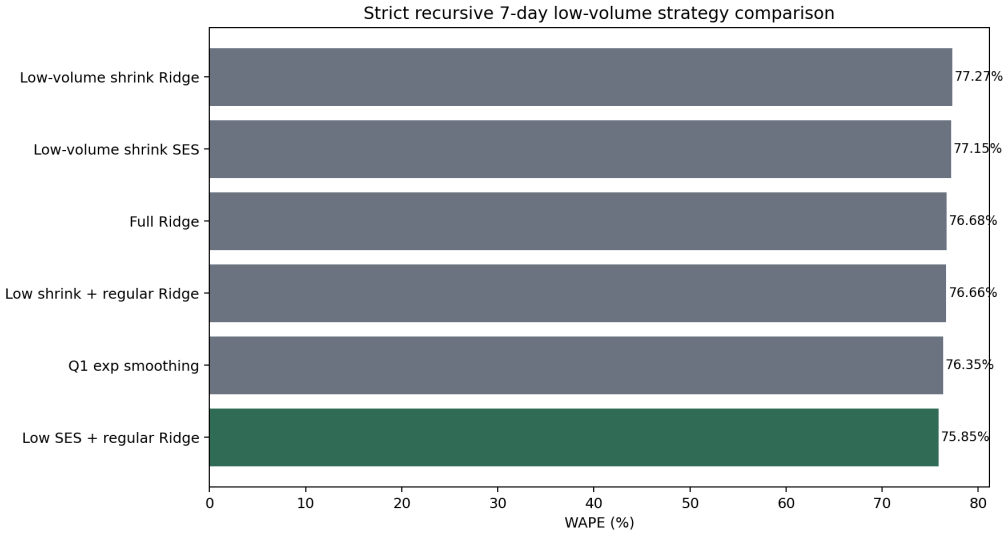



图 13: 低销量鲁棒性策略 WAPE 对比，用于说明混合策略的验证效果

## 8.4 未来 7 天预测与整数化

未来 7 天天气、温度、风力和活动日没有附件真实观测，最终预测使用历史同期参考情景；清明节日历按 2022-04-03 至 2022-04-05 为节假日、2022-04-02 为调休上班日处理。因此，最终预测为给定外部变量情景下的条件预测。

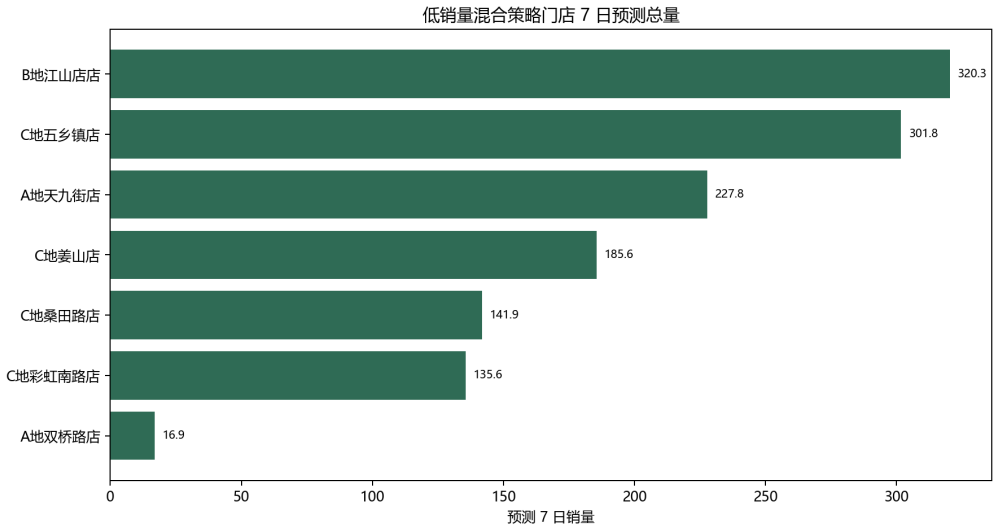



图 14: 低销量混合策略下未来 7 天门店预测总量，用于展示推荐预测表在门店层面的分布

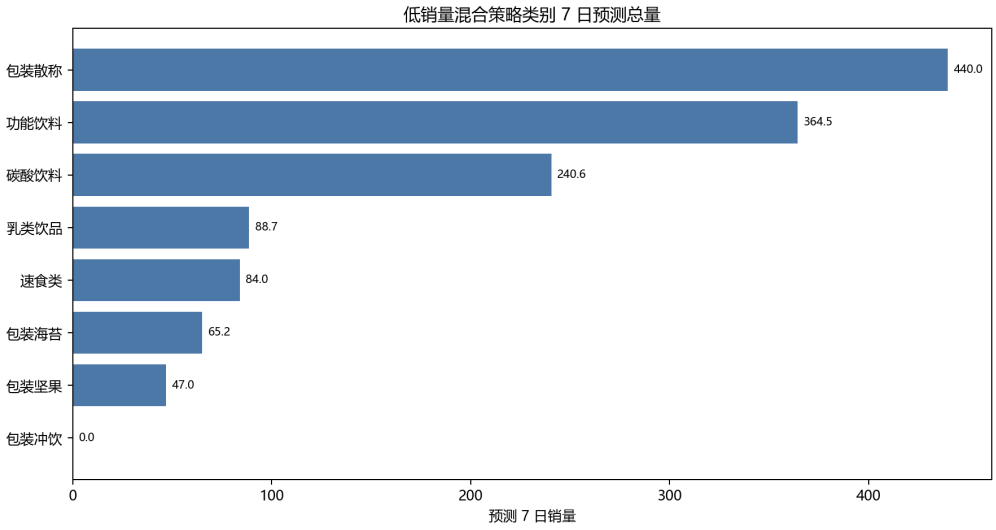



图 15: 低销量混合策略下未来 7 天类别预测总量，用于展示推荐预测表在类别层面的分布

未来 7 天低销量混合策略条件预测中，各门店预测总销量最高的是 B 地江山店店 320.342，其次为 C 地五乡镇店 301.763 和 A 地天九街店 227.846；各类别预测最高的是包装散称 439.965，其次为功能饮料 364.481 和碳酸饮料 240.589。图 14 和图 15 均由同一混合策略预测表生成，与下述预测总量一致。完整门店–商品–日期小数预测表见 `outputs/final_7day_forecast_hybrid_low_volume.csv`；原完整 Ridge 预测表 `outputs/final_7day_forecast.csv` 保留为对照。

由于实际销量以件数计，提交展示时对模型预测值进行非负约束和四舍五入整数化处理：预测值小于 0 的先置为 0，极小预测值低于 $10^{-6}$ 时置为 0，其余按四舍五

入转为整数；原始小数预测结果仍作为模型连续输出保留。混合策略整数化预测表为 `outputs/final_7day_forecast_hybrid_low_volume_integer.csv`。混合策略小数预测表 7 天预测总量为 1329.894，整数化后 7 天预测总量为 1328；原完整 Ridge 小数预测总量为 1311.294。活动日变量与天气变量一样属于历史同期情景假设，不代表已知的真实未来促销安排。

## 8.5 误差诊断与情景敏感性

天气敏感性检验 `outputs/q4_weather_sensitivity_report.md` 显示，完整外部变量 Ridge 的严格递推 WAPE 为 76.68%，去除天气变量 Ridge 为 77.73%，历史同期天气情景 Ridge 为 76.57%。因此，天气变量可作为补充情景特征，但不能写成稳定因果因素。

误差诊断 `outputs/q4_error_diagnosis_report.md` 显示，综合 Ridge 在门店 A 地双桥路店、低销量商品和样本不足的活动日场景误差较大；间歇性需求审查 `outputs/intermittent_demand_review.md` 显示，76 条门店–商品序列中有 54 条零销量比例不低于 50%，解释了门店–商品层级 WAPE 较高的原因。间歇性需求建模思想可参考 Croston 方法 [2]，但本文不将其作为主模型。

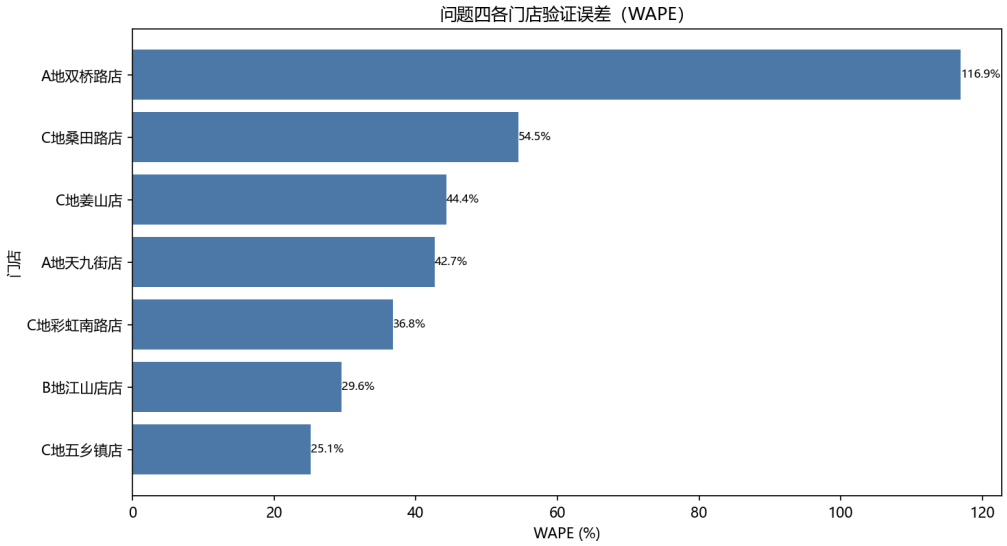



图 16: 门店误差诊断，用于定位严格递推验证下误差较高的门店

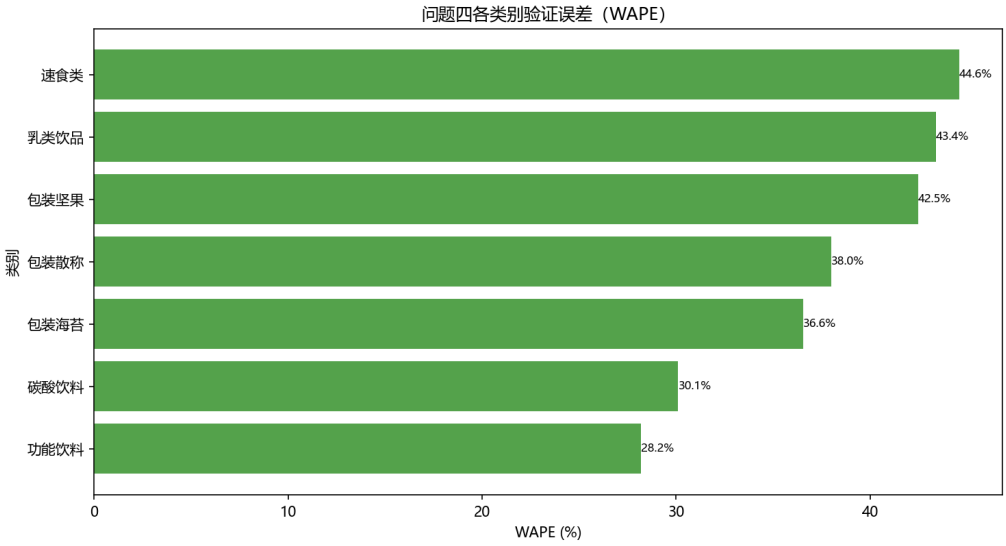



图 17: 类别误差诊断，用于定位严格递推验证下误差较高的类别

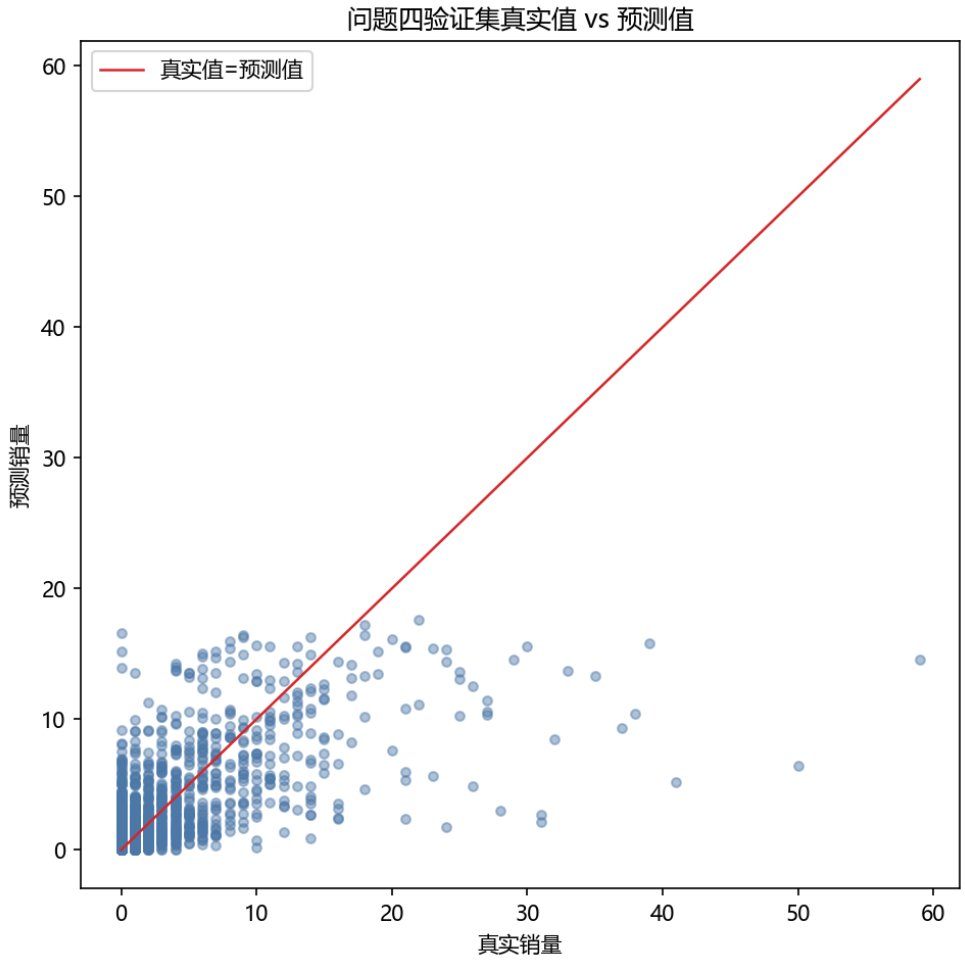



图 18: 严格递推验证真实值与预测值散点，用于检查预测值与真实值的对应关系

# 9 模型评价

## 9.1 误差指标

本文主要采用 MAE、RMSE 和 WAPE 评价预测误差：

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \tag{11}$$

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \tag{12}$$

$$WAPE = \frac{\sum_{i=1}^{n}|y_i - \hat{y}_i|}{\sum_{i=1}^{n}|y_i|}. \tag{13}$$

MAE 表示平均绝对偏差，便于解释平均误差规模；RMSE 对大误差更敏感，适合检查极端预测偏差；WAPE 用总绝对误差除以总实际销量，适合本题中不同门店和商品销量规模差异较大的情况。对于真实销量为 0 的类别或商品，WAPE 无法定义，需要结合 MAE 和预测合计单独说明。

## 9.2 分问题评价

问题一中，日滚动验证结果显示，门店层面移动平均 (7 日) WAPE=37.49%，商品层面简单指数平滑 WAPE=38.14%，门店–商品层面简单指数平滑 WAPE=74.95%。补充的 7 日窗口总量验证表明，原问题一主方案在商品层面 WAPE=21.51%，门店层面 WAPE=22.12%，门店–商品层面 WAPE=43.06%。因此，问题一保留简单可解释模型，不引入复杂模型替换主方案。

问题二中，类别聚合后直接预测 + 简单指数平滑的 WAPE=34.74%，略优于单品预测后按类别加总的 WAPE=34.83%。销量规模分组 WAPE=24.24%，相关性聚类 WAPE=31.02%，但二者不直接回答“同类零食类别预测”要求，因此只作为补充，不作为主方案。

问题三中，合并天气、对数销量和历史控制固定效应回归的验证 WAPE=84.53%，优于原详细天气固定效应回归的 WAPE=88.09%。随机森林 grouped weather + history 的 WAPE=84.66%，可作为预测贡献补充，但不提供清晰方向，也不能解释因果。问题三主结论应写成“条件统计关联”。

问题四中，严格 7 日递推验证是主评价口径。该口径下，问题一门店–商品简单指数平滑 WAPE=76.35%，综合 Ridge WAPE=76.68%，14 日滚动均值基准模型 WAPE=77.66%，7 日移动平均基准模型 WAPE=78.43%。综合 Ridge 优于问题四内部基准模型，但未超过问题一强基准模型。因此，本文先将其定位为融合前三问信息的解释型综合模型。进一步的低销量鲁棒性检验显示，低销量序列使用指数平滑、常规序列使用综合 Ridge 的混合策略 WAPE=75.85%，相对问题一强基准降低 0.49 个百分

点；该结果说明细粒度稀疏需求需要分层处理，但由于改进幅度有限，仍应克制表述为稳健性改进。本轮有限优化还比较了不同低销量阈值、近 28 日均值兜底和层级比例校准；虽然近 28 日均值候选在同一验证集上 WAPE=74.95%，但留一窗口选模平均改善仅 0.19 个百分点，未达到稳定替换阈值，因此不替换当前推荐预测表。

## 9.3 不确定性

不确定性主要来自四类来源。第一，未来天气和活动日采用历史同期情景，若实际天气或促销安排偏离该情景，最终预测会变化。第二，低销量门店–商品序列存在大量零销量，当前混合策略下低销量序列整体 WAPE 约为 122.53%，少量绝对误差即可被低销量分母放大；混合策略虽有所改善，但并未消除稀疏需求带来的不确定性。第三，部分稀有天气样本天数较少，例如雨夹雪、暴雨等，不能把其系数作为稳定主结论。第四，严格递推验证只有 4 个完整 7 日窗口，模型选择结论仍需谨慎。

为辅助解释最终点预测，本文基于当前混合策略在 4 个严格 7 日递推验证窗口上的 7 日聚合残差构造经验区间。对每个门店、商品或类别，先计算“真实 7 日销量减预测 7 日销量”的残差，再取历史残差的 10% 和 90% 分位数加到未来点预测上，形成经验 80% 残差区间。该区间不是严格置信区间，只用于展示历史递推误差下的不确定性范围。门店级结果见图 19，完整表见 `tables/final_forecast_interval_by_store.csv`、`tables/final_forecast_interval_by_product.csv` 和 `tables/final_forecast_interval_by_category.csv`。

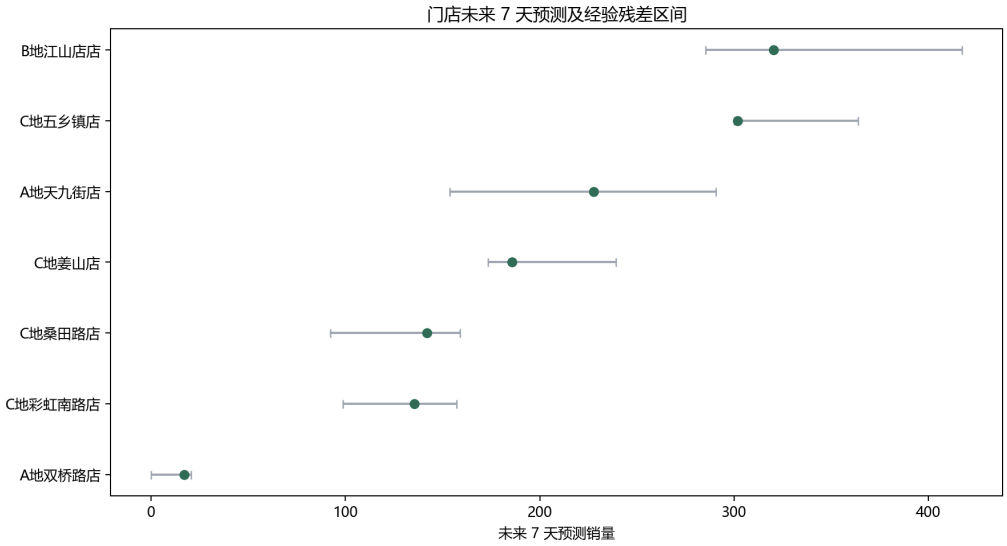



图 19: 基于严格递推残差的门店未来 7 天经验预测区间，用于提示点预测的不确定性

# 10 优缺点分析与结论

## 10.1 模型优点

本文模型具有以下优点：

1. 数据口径清晰，区分正向销量、净销量和负向调整。
2. 模型与题目四问一一对应，形成门店、商品、类别和门店–商品的分层预测框架。
3. 主模型优先采用移动平均、同星期均值、指数平滑和固定效应回归，公式简单、可解释、易复现。
4. 对综合模型进行了信息泄露审查，并以严格 7 日递推验证作为主口径。
5. 对外部变量和商品关联均设置了证据边界，没有把相关性写成因果性。

## 10.2 模型局限

模型局限包括：第一，门店–商品粒度存在大量零销量和间歇性需求，导致 WAPE 较高；第二，未来真实天气和活动安排不可得，含外部变量预测依赖历史同期情景；第三，附件没有顾客购物篮、价格实验、库存和陈列信息，因此无法识别商品互补、替代或促销因果效应；第四，验证期只有有限的完整 7 日窗口，显著性检验和稳定性判断应谨慎。

## 10.3 结论

本文完成了休闲零食连锁店商品销量预测的四问建模。问题一保留移动平均、同星期均值和简单指数平滑，建立了门店、商品和门店–商品的基础预测；问题二以附件类别字段整合同类零食，商品相关性分析作为同步波动证据，类别预测主方案为类别聚合后直接预测；问题三采用合并天气、对数销量和历史控制固定效应回归，得到外部变量与销量之间的条件统计关联；问题四采用综合 Ridge 模型融合前三问信息，并用严格 7 日递推验证检验未来 7 天预测能力。

最终结论是：简单可解释模型在本题中具有很强竞争力。问题四综合 Ridge 虽然在日滚动一步预测和部分聚合层级上有数值优势，但在严格 7 日递推主粒度下未超过问题一门店–商品简单指数平滑强基准模型。因此，本文不夸大综合模型的预测提升，而将其作为融合历史销量、类别和外部变量的解释框架。进一步的低销量鲁棒性检验表明，对低销量组合采用指数平滑、对常规组合采用综合 Ridge 的混合策略在严格递推主粒度上 WAPE=75.85%，略优于两个单一模型。有限优化中近 28 日均值兜底候选在同一验证集上表现更低，但留一窗口检查未达到稳定替换阈值，因此本文仍推荐使用 `outputs/final_7day_forecast_hybrid_low_volume.csv` 作为候选最终预测表，同时保留 `outputs/final_7day_forecast.csv` 作为完整 Ridge 对照；两者均是在历史同期天气和活动日情景下的条件预测。混合策略整数化提交副本为 `outputs/final_7day_forecast_hybrid_low_volume_integer.csv`。

# 参考文献


[1] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.

[2] J. D. Croston. Forecasting and stock control for intermittent demands. *Operational Research Quarterly*, 23(3):289–303, 1972.

[3] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[4] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 3 edition, 2021.

[5] Douglas C. Montgomery, Elizabeth A. Peck, and G. Geoffrey Vining. *Introduction to Linear Regression Analysis*. Wiley, 5 edition, 2012.


# A 附录

## A.1 主要代码文件

- 数据读取与清洗：`src/data_loader.py`、`src/preprocessing.py`
- 特征工程：`src/features.py`
- 问题二分析：`src/stage3_q2_analysis.py`
- 问题三分析：`src/stage4_q3_analysis.py`
- 问题四综合模型：`src/stage5_q4_analysis.py`
- 严格递推验证：`src/stage5_q4_recursive_validation.py`
- 问题四误差诊断：`src/stage5_q4_error_diagnosis.py`
- 天气敏感性分析：`src/stage5_q4_weather_sensitivity.py`
- 严格递推消融检验：`src/stage5_q4_ablation.py`
- 低销量鲁棒性策略：`src/stage5_q4_low_volume_strategy.py`
- 低销量混合最终预测：`src/stage5_q4_hybrid_final_forecast.py`
- 候选优化、不确定性和区间分析：`src/stage5_q4_award_iteration.py`
- 间歇性需求审查：`src/intermittent_demand_review.py`

## A.2 主要结果文件

- 数据检查：`outputs/stage1_data_inspection_report.md`
- 问题一报告：`outputs/stage2_q1_modeling_report.md`
- 问题二报告：`outputs/stage3_q2_relationship_report.md`
- 问题三报告：`outputs/stage4_q3_factor_analysis_report.md`

- 问题四报告：outputs/stage5_q4_final_model_report.md
- 信息泄露审查：outputs/q4_information_leakage_review.md
- 严格递推验证：outputs/q4_recursive_7day_model_comparison.md
- 误差诊断：outputs/q4_error_diagnosis_report.md
- 天气敏感性检验：outputs/q4_weather_sensitivity_report.md
- 严格递推消融检验：outputs/q4_ablation_report.md
- 低销量鲁棒性策略：outputs/q4_low_volume_strategy_report.md
- 混合策略最终预测说明：outputs/q4_hybrid_final_forecast_report.md
- 候选优化复核：outputs/q4_candidate_model_review.md
- 预测不确定性报告：outputs/forecast_uncertainty_report.md
- 增强误差归因：outputs/q4_error_attribution_enhanced_report.md
- 间歇性需求审查：outputs/intermittent_demand_review.md
- 最终方法选择：outputs/method_search/final_method_selection.md
- 小数预测表：outputs/final_7day_forecast.csv
- 整数预测表：outputs/final_7day_forecast_integer.csv
- 混合策略小数预测表：outputs/final_7day_forecast_hybrid_low_volume.csv
- 混合策略整数预测表：outputs/final_7day_forecast_hybrid_low_volume_integer.csv

## A.3 核心 Python 代码

以下列出本次论文最终结论直接依赖的核心新增代码。完整项目代码保存在项目仓库中。复现问题四候选最终结果时，应先运行严格递推消融检验生成 tables/q4_ablation_predictions.csv，再运行低销量策略检验、混合预测生成和候选优化脚本。核心依赖包括 src/features.py、src/models.py、src/evaluation.py 以及 data/processed/modeling_base_table.csv；附录只展示关键阶段脚本，完整可复现代码以项目仓库的 src/ 目录为准。

Listing 1: 严格 7 日递推消融检验代码

```python
from __future__ import annotations

import json
import sys
from dataclasses import dataclass
from datetime import date
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
```

```
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.evaluation import calculate_metrics
from src.features import add_leakage_safe_sales_features,
    build_store_product_panel
from src.models import (
    build_ridge_forecaster,
    clipped_predict,
    exp_smoothing_prediction,
    optimize_exp_smoothing_alpha,
)

TABLES = ROOT / "tables"
OUTPUTS = ROOT / "outputs"
FIGURES = ROOT / "figures"

TARGET = "positive_sales"
WINDOW_LENGTH = 7
VALIDATION_STARTS = pd.to_datetime(
    ["2022-03-01", "2022-03-08", "2022-03-15", "2022-03-22"]
)

HISTORY_NUMERIC = [
    "lag_1",
    "lag_7",
    "lag_14",
    "rolling_mean_7",
    "rolling_mean_14",
    "rolling_std_7",
]
CALENDAR_NUMERIC = ["weekday", "month", "is_weekend"]
EVENT_NUMERIC = ["is_holiday", "is_activity_day"]
WEATHER_NUMERIC = ["max_temperature", "min_temperature", "wind_power"]
ID_CATEGORICAL = ["store_id_str", "product_id_str"]
CATEGORY_CATEGORICAL = ["category"]
WEATHER_CATEGORICAL = ["weather"]

```

```
ALL_NUMERIC = HISTORY_NUMERIC + CALENDAR_NUMERIC + EVENT_NUMERIC +
    WEATHER_NUMERIC
ALL_CATEGORICAL = ID_CATEGORICAL + CATEGORY_CATEGORICAL + WEATHER_CATEGORICAL


@dataclass(frozen=True)
class AblationSpec:
    variant: str
    model_label: str
    feature_scope: str
    numeric_features: list[str]
    categorical_features: list[str]


SPECS = [
    AblationSpec(
        variant="ridge_full_external",
        model_label="完整综合Ridge",
        feature_scope="历史滞后 + 日历 + 节假日/活动日 + 天气 + 门店/商品/类别",
        numeric_features=ALL_NUMERIC,
        categorical_features=ALL_CATEGORICAL,
    ),
    AblationSpec(
        variant="ridge_no_weather",
        model_label="去天气Ridge",
        feature_scope="去除天气、温度、风力，保留历史、日历、节假日/活动日和门店/
            商品/类别",
        numeric_features=HISTORY_NUMERIC + CALENDAR_NUMERIC + EVENT_NUMERIC,
        categorical_features=ID_CATEGORICAL + CATEGORY_CATEGORICAL,
    ),
    AblationSpec(
        variant="ridge_no_activity",
        model_label="去活动日Ridge",
        feature_scope="去除活动日，保留历史、日历、节假日、天气和门店/商品/类别",
        numeric_features=HISTORY_NUMERIC
        + CALENDAR_NUMERIC
        + ["is_holiday"]
        + WEATHER_NUMERIC,
        categorical_features=ALL_CATEGORICAL,
    ),
```

```
    AblationSpec(
        variant="ridge_no_event",
        model_label="去节假日活动Ridge",
        feature_scope="去除节假日和活动日，保留历史、日历、天气和门店/商品/类别",
        numeric_features=HISTORY_NUMERIC + CALENDAR_NUMERIC + WEATHER_NUMERIC,
        categorical_features=ALL_CATEGORICAL,
    ),
    AblationSpec(
        variant="ridge_history_id_category",
        model_label="仅历史与层级Ridge",
        feature_scope="仅使用历史滞后/滚动特征和门店/商品/类别",
        numeric_features=HISTORY_NUMERIC,
        categorical_features=ID_CATEGORICAL + CATEGORY_CATEGORICAL,
    ),
    AblationSpec(
        variant="ridge_history_calendar",
        model_label="历史日历Ridge",
        feature_scope="历史滞后/滚动 + 星期/月/周末 + 门店/商品/类别",
        numeric_features=HISTORY_NUMERIC + CALENDAR_NUMERIC,
        categorical_features=ID_CATEGORICAL + CATEGORY_CATEGORICAL,
    ),
    AblationSpec(
        variant="ridge_no_history",
        model_label="无销量历史Ridge",
        feature_scope="不使用 lag/rolling，仅使用日历、外部变量和门店/商品/类别",
        numeric_features=CALENDAR_NUMERIC + EVENT_NUMERIC + WEATHER_NUMERIC,
        categorical_features=ALL_CATEGORICAL,
    ),
]


def df_to_md(df: pd.DataFrame, max_rows: int | None = None, digits: int = 3)
    -> str:
    if df is None or df.empty:
        return "无。"
    out = df.copy()
    if max_rows is not None:
        out = out.head(max_rows)
    for col in out.columns:
        if pd.api.types.is_float_dtype(out[col]):
```

```
            out[col] = out[col].map(lambda x: "" if pd.isna(x) else f"{x:.{digits}f}")
    cols = list(out.columns)
    lines = [
        "| " + " | ".join(cols) + " |",
        "| " + " | ".join(["---"] * len(cols)) + " |",
    ]
    for _, row in out.iterrows():
        lines.append(
            "| "
            + " | ".join("" if pd.isna(row[col]) else str(row[col]) for col in cols)
            + " |"
        )
    return "\n".join(lines)


def prepend_result_log(row: str) -> None:
    path = ROOT / "RESULT_LOG.md"
    text = path.read_text(encoding="utf-8")
    if row in text:
        return
    lines = text.splitlines()
    insert_idx = None
    for idx, line in enumerate(lines):
        if line.startswith("|------"):
            insert_idx = idx + 1
            break
    if insert_idx is not None:
        lines.insert(insert_idx, row)
        path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def save_csv(df: pd.DataFrame, name: str) -> None:
    df.to_csv(TABLES / name, index=False, encoding="utf-8-sig")
    df.to_csv(OUTPUTS / name, index=False, encoding="utf-8-sig")


def prepare_model_frame(panel: pd.DataFrame) -> pd.DataFrame:
    out = add_leakage_safe_sales_features(panel, TARGET)
```

```
    out["store_id_str"] = out["store_id"].astype(str)
    out["product_id_str"] = out["product_id"].astype(str)
    out["category"] = out["category"].astype(str)
    out["weather"] = out["weather"].fillna("未知").astype(str)
    for col in ALL_NUMERIC:
        out[col] = pd.to_numeric(out[col], errors="coerce").fillna(0.0)
    out["has_external_data"] = out["has_external_data"].fillna(0).astype(int)
    return out.sort_values(["store_id", "product_id", "date"]).reset_index(
        drop=True)


def validation_windows(feature_df: pd.DataFrame) -> tuple[list[tuple[pd.
    Timestamp, pd.Timestamp]], list[dict]]:
    external_dates = set(
        pd.to_datetime(
            feature_df.loc[feature_df["has_external_data"] == 1, "date"]
        ).dt.normalize()
    )
    data_dates = set(pd.to_datetime(feature_df["date"]).dt.normalize())
    windows: list[tuple[pd.Timestamp, pd.Timestamp]] = []
    excluded: list[dict] = []
    for start in VALIDATION_STARTS:
        end = start + pd.Timedelta(days=WINDOW_LENGTH - 1)
        dates = pd.date_range(start, end, freq="D")
        has_all_sales = all(d in data_dates for d in dates)
        has_all_external = all(d in external_dates for d in dates)
        if has_all_sales and has_all_external:
            windows.append((start, end))
        else:
            excluded.append(
                {
                    "window_start": start.date().isoformat(),
                    "window_end": end.date().isoformat(),
                    "reason": "不足完整 7 日销售或外部变量覆盖，未纳入消融检验",
                }
            )
    return windows, excluded


def history_feature_row(values: list[float]) -> dict[str, float]:
```

```
    values = [float(v) for v in values if pd.notna(v)]
    recent7 = values[-7:] if values else [0.0]
    recent14 = values[-14:] if values else [0.0]
    return {
        "lag_1": float(values[-1]) if len(values) >= 1 else 0.0,
        "lag_7": float(values[-7]) if len(values) >= 7 else 0.0,
        "lag_14": float(values[-14]) if len(values) >= 14 else 0.0,
        "rolling_mean_7": float(np.mean(recent7)),
        "rolling_mean_14": float(np.mean(recent14)),
        "rolling_std_7": float(np.std(recent7, ddof=1)) if len(recent7) >= 2
            else 0.0,
    }


def build_lookup_tables(panel: pd.DataFrame, feature_df: pd.DataFrame) ->
    tuple[pd.DataFrame, dict, dict]:
    combos = (
        panel[["store_id", "store_name", "product_id", "product_name", "
            category"]]
        .drop_duplicates()
        .sort_values(["store_id", "product_id"])
        .reset_index(drop=True)
    )
    actual_lookup = (
        panel.set_index(["date", "store_id", "product_id"])[TARGET].astype(
            float).to_dict()
    )
    external_cols = [
        "date",
        "weather",
        "max_temperature",
        "min_temperature",
        "wind_power",
        "is_holiday",
        "is_activity_day",
        "weekday",
        "month",
        "is_weekend",
        "has_external_data",
    ]
```

```
    external_lookup = (
        feature_df[external_cols].drop_duplicates("date").set_index("date").
            to_dict("index")
    )
    return combos, actual_lookup, external_lookup


def initial_history(panel: pd.DataFrame, combos: pd.DataFrame, window_start:
    pd.Timestamp) -> dict:
    before = panel[panel["date"] < window_start]
    history = {}
    for row in combos.itertuples():
        key = (int(row.store_id), int(row.product_id))
        history[key] = (
            before[
                (before["store_id"] == row.store_id)
                & (before["product_id"] == row.product_id)
            ]
            .sort_values("date")[TARGET]
            .astype(float)
            .tolist()
        )
    return history


def batch_rows_for_date(
    combos: pd.DataFrame,
    history: dict,
    target_date: pd.Timestamp,
    ext: dict,
) -> pd.DataFrame:
    rows = []
    for row in combos.itertuples():
        rows.append(
            {
                "date": pd.Timestamp(target_date),
                "store_id": int(row.store_id),
                "store_name": row.store_name,
                "product_id": int(row.product_id),
                "product_name": row.product_name,
```

```
277                 "category": str(row.category),
278                 **history_feature_row(history[(int(row.store_id), int(row.
                        product_id))]),
279                 "weekday": int(ext["weekday"]),
280                 "month": int(ext["month"]),
281                 "is_weekend": int(ext["is_weekend"]),
282                 "is_holiday": int(ext["is_holiday"]),
283                 "is_activity_day": int(ext["is_activity_day"]),
284                 "max_temperature": float(ext.get("max_temperature", 0.0)),
285                 "min_temperature": float(ext.get("min_temperature", 0.0)),
286                 "wind_power": float(ext.get("wind_power", 0.0)),
287                 "weather": str(ext.get("weather", "未知")),
288                 "store_id_str": str(row.store_id),
289                 "product_id_str": str(row.product_id),
290             }
291         )
292     return pd.DataFrame(rows)
293
294
295 def append_prediction_rows(
296     rows: list[dict],
297     batch: pd.DataFrame,
298     actual_lookup: dict,
299     variant: str,
300     model_label: str,
301     feature_scope: str,
302     window_start: pd.Timestamp,
303 ) -> None:
304     for pred_row in batch.itertuples():
305         key = (pd.Timestamp(pred_row.date), pred_row.store_id, pred_row.
                product_id)
306         actual = float(actual_lookup.get(key, 0.0))
307         rows.append(
308             {
309                 "validation_mode": "strict_recursive_7day",
310                 "window_start": window_start,
311                 "window_end": window_start + pd.Timedelta(days=WINDOW_LENGTH -
                        1),
312                 "date": pd.Timestamp(pred_row.date),
313                 "horizon": int((pd.Timestamp(pred_row.date) - window_start).days
```

```
                    + 1),
314                 "store_id": pred_row.store_id,
315                 "store_name": pred_row.store_name,
316                 "product_id": pred_row.product_id,
317                 "product_name": pred_row.product_name,
318                 "category": pred_row.category,
319                 "actual": actual,
320                 "prediction": float(pred_row.prediction),
321                 "model": variant,
322                 "model_label": model_label,
323                 "feature_scope": feature_scope,
324                 "feature_source": "验证窗口内 lag/rolling 使用前序预测值递推生成",
325             }
326         )
327
328
329 def validate_exp_smoothing(
330     panel: pd.DataFrame,
331     feature_df: pd.DataFrame,
332     windows: list[tuple[pd.Timestamp, pd.Timestamp]],
333 ) -> pd.DataFrame:
334     combos, actual_lookup, external_lookup = build_lookup_tables(panel,
        feature_df)
335     rows: list[dict] = []
336     variant = "q1_store_product_exp_smoothing"
337     label = "问题一门店-商品简单指数平滑"
338     for window_start, window_end in windows:
339         history = initial_history(panel, combos, window_start)
340         alphas = {
341             (int(row.store_id), int(row.product_id)):
                optimize_exp_smoothing_alpha(
342                 pd.Series(history[(int(row.store_id), int(row.product_id))],
                    dtype=float)
343             )
344             for row in combos.itertuples()
345         }
346         for target_date in pd.date_range(window_start, window_end, freq="D"):
347             batch_rows = []
348             ext = external_lookup[pd.Timestamp(target_date)]
349             for row in combos.itertuples():
```

```
            key = (int(row.store_id), int(row.product_id))
            pred = max(0.0, float(exp_smoothing_prediction(history[key],
                alphas[key])))
            batch_rows.append(
                {
                    "date": pd.Timestamp(target_date),
                    "store_id": int(row.store_id),
                    "store_name": row.store_name,
                    "product_id": int(row.product_id),
                    "product_name": row.product_name,
                    "category": str(row.category),
                    "weather": str(ext.get("weather", "未知")),
                    "prediction": pred,
                }
            )
        batch = pd.DataFrame(batch_rows)
        append_prediction_rows(
            rows,
            batch,
            actual_lookup,
            variant,
            label,
            "仅使用该门店-商品自身历史销量，不使用外部变量",
            window_start,
        )
        for pred_row in batch.itertuples():
            history[(int(pred_row.store_id), int(pred_row.product_id))].
                append(
                float(pred_row.prediction)
            )
    return pd.DataFrame(rows)


def validate_ridge_spec(
    panel: pd.DataFrame,
    feature_df: pd.DataFrame,
    windows: list[tuple[pd.Timestamp, pd.Timestamp]],
    spec: AblationSpec,
) -> pd.DataFrame:
    combos, actual_lookup, external_lookup = build_lookup_tables(panel,
```

```
        feature_df)
    rows: list[dict] = []
    features = spec.numeric_features + spec.categorical_features
    for window_start, window_end in windows:
        train = feature_df[
            (feature_df["date"] < window_start) & (feature_df["
                has_external_data"] == 1)
        ].copy()
        model = build_ridge_forecaster(
            spec.numeric_features,
            spec.categorical_features,
            alpha=10.0,
        )
        model.fit(train[features], train[TARGET])

        history = initial_history(panel, combos, window_start)
        for target_date in pd.date_range(window_start, window_end, freq="D"):
            ext = external_lookup[pd.Timestamp(target_date)]
            batch = batch_rows_for_date(combos, history, pd.Timestamp(
                target_date), ext)
            for col in spec.numeric_features:
                batch[col] = pd.to_numeric(batch[col], errors="coerce").fillna
                    (0.0)
            batch["prediction"] = clipped_predict(model, batch[features])
            append_prediction_rows(
                rows,
                batch,
                actual_lookup,
                spec.variant,
                spec.model_label,
                spec.feature_scope,
                window_start,
            )
            for pred_row in batch.itertuples():
                history[(int(pred_row.store_id), int(pred_row.product_id))].
                    append(
                    float(pred_row.prediction)
                )
    return pd.DataFrame(rows)

```

```
423
424 def aggregate_predictions(preds: pd.DataFrame, level: str) -> pd.DataFrame:
425     if level == "store_product":
426         return preds.copy()
427     if level == "store":
428         group_cols = ["date", "window_start", "model", "model_label", "store_id
            ", "store_name"]
429     elif level == "product":
430         group_cols = [
431             "date",
432             "window_start",
433             "model",
434             "model_label",
435             "product_id",
436             "product_name",
437             "category",
438         ]
439     elif level == "category":
440         group_cols = ["date", "window_start", "model", "model_label", "category
            "]
441     else:
442         raise ValueError(level)
443     return (
444         preds.groupby(group_cols, dropna=False, as_index=False)
445         .agg(actual=("actual", "sum"), prediction=("prediction", "sum"))
446         .sort_values(group_cols)
447     )
448
449
450 def metric_table(preds: pd.DataFrame, windows: list[tuple[pd.Timestamp, pd.
    Timestamp]]) -> pd.DataFrame:
451     validation_start = min(start for start, _ in windows).date().isoformat()
452     validation_end = max(end for _, end in windows).date().isoformat()
453     rows = []
454     for level in ["store_product", "store", "product", "category"]:
455         metric_input = aggregate_predictions(preds, level)
456         for (model, label), group in metric_input.groupby(["model", "
            model_label"], dropna=False):
457             metrics = calculate_metrics(group["actual"], group["prediction"])
458             feature_scope = (
```

```
            preds.loc[preds["model"] == model, "feature_scope"].dropna().
                iloc[0]
        )
        rows.append(
            {
                "validation_mode": "strict_recursive_7day",
                "validation_start": validation_start,
                "validation_end": validation_end,
                "level": level,
                "model": model,
                "model_label": label,
                "feature_scope": feature_scope,
                **metrics,
                "WAPE_pct": metrics["WAPE"] * 100,
                "actual_sum": float(group["actual"].sum()),
                "prediction_sum": float(group["prediction"].sum()),
                "bias": float(group["prediction"].sum() - group["actual"].sum
                    ()),
                "bias_pct_of_actual": float(
                    (group["prediction"].sum() - group["actual"].sum())
                    / group["actual"].sum()
                )
                if group["actual"].sum() != 0
                else np.nan,
                "n": int(len(group)),
            }
        )
    out = pd.DataFrame(rows)
    sp = out[out["level"] == "store_product"].set_index("model")
    full_wape = float(sp.loc["ridge_full_external", "WAPE_pct"])
    q1_wape = float(sp.loc["q1_store_product_exp_smoothing", "WAPE_pct"])
    out["WAPE_pct_point_change_vs_full"] = out["WAPE_pct"] - full_wape
    out["WAPE_pct_point_change_vs_q1"] = out["WAPE_pct"] - q1_wape
    return out.sort_values(["level", "WAPE", "MAE", "RMSE"]).reset_index(drop=
        True)


def plot_store_product_wape(metrics: pd.DataFrame) -> None:
    sp = metrics[metrics["level"] == "store_product"].sort_values("WAPE_pct")
    label_map = {
```

```
        "q1_store_product_exp_smoothing": "Q1 exp smoothing",
        "ridge_full_external": "Full Ridge",
        "ridge_no_weather": "No weather",
        "ridge_no_activity": "No activity",
        "ridge_no_event": "No holiday/activity",
        "ridge_history_id_category": "History + hierarchy",
        "ridge_history_calendar": "History + calendar",
        "ridge_no_history": "No sales history",
    }
    labels = [label_map.get(model, model) for model in sp["model"]]
    values = sp["WAPE_pct"].tolist()
    plt.figure(figsize=(10, 5.8))
    colors = ["#2F6B55" if m == "q1_store_product_exp_smoothing" else "#6B7280
        " for m in sp["model"]]
    plt.barh(labels, values, color=colors)
    plt.xlabel("WAPE (%)")
    plt.title("Q4 strict recursive 7-day ablation at store-product level")
    for idx, value in enumerate(values):
        plt.text(value + 0.25, idx, f"{value:.2f}%", va="center", fontsize=9)
    plt.tight_layout()
    plt.savefig(FIGURES / "q4_ablation_store_product_wape.png", dpi=180)
    plt.close()


def build_report(
    windows: list[tuple[pd.Timestamp, pd.Timestamp]],
    excluded: list[dict],
    metrics: pd.DataFrame,
) -> str:
    window_df = pd.DataFrame(
        [
            {
                "window_start": start.date().isoformat(),
                "window_end": end.date().isoformat(),
                "length": int((end - start).days + 1),
            }
            for start, end in windows
        ]
    )
    spec_df = pd.DataFrame(
```

```
        [
            {
                "model": "q1_store_product_exp_smoothing",
                "model_label": "问题一门店-商品简单指数平滑",
                "feature_scope": "仅使用该门店-商品自身历史销量，不使用外部变量",
            }
        ]
        + [
            {
                "model": spec.variant,
                "model_label": spec.model_label,
                "feature_scope": spec.feature_scope,
            }
            for spec in SPECS
        ]
    )
    sp = metrics[metrics["level"] == "store_product"].copy()
    sp_table = sp[
        [
            "model_label",
            "feature_scope",
            "MAE",
            "RMSE",
            "WAPE_pct",
            "WAPE_pct_point_change_vs_full",
            "WAPE_pct_point_change_vs_q1",
            "actual_sum",
            "prediction_sum",
            "bias_pct_of_actual",
            "n",
        ]
    ].copy()
    level_table = metrics[
        ["level", "model_label", "MAE", "RMSE", "WAPE_pct", "bias_pct_of_actual
            ", "n"]
    ].copy()
    best = sp.sort_values(["WAPE", "MAE", "RMSE"]).iloc[0]
    full = sp[sp["model"] == "ridge_full_external"].iloc[0]
    no_weather = sp[sp["model"] == "ridge_no_weather"].iloc[0]
    no_activity = sp[sp["model"] == "ridge_no_activity"].iloc[0]
```

```
    history_calendar = sp[sp["model"] == "ridge_history_calendar"].iloc[0]
    no_history = sp[sp["model"] == "ridge_no_history"].iloc[0]
    q1 = sp[sp["model"] == "q1_store_product_exp_smoothing"].iloc[0]

    weather_gain = no_weather["WAPE_pct"] - full["WAPE_pct"]
    activity_gain = no_activity["WAPE_pct"] - full["WAPE_pct"]
    history_penalty = no_history["WAPE_pct"] - full["WAPE_pct"]
    calendar_gap = history_calendar["WAPE_pct"] - full["WAPE_pct"]
    ridge_vs_q1 = full["WAPE_pct"] - q1["WAPE_pct"]

    return f"""# 问题四严格 7 日递推消融实验报告

执行日期: {date.today().isoformat()}

## 1. 实验目的

本实验不继续盲目调参，而是在与问题四主验证一致的严格 7 日递推口径下，检查综合
    Ridge 模型中历史销量、日历、节假日/活动日、天气、门店/商品/类别等信息分别提供
    了多少可复现增益。每个验证窗口开始时只使用窗口开始日前真实销量，窗口内第 2 天
    及以后 `lag` 和 `rolling` 均由前序预测值递推生成。

## 2. 验证窗口

{df_to_md(window_df)}

未纳入窗口:

{df_to_md(pd.DataFrame(excluded))}

## 3. 消融模型设置

{df_to_md(spec_df, 20)}

## 4. 门店-商品主粒度结果

{df_to_md(sp_table, 20)}

完整分层结果见 `tables/q4_ablation_metrics.csv`，逐日预测明细见 `tables/
    q4_ablation_predictions.csv`。图表已保存至 `figures/
    q4_ablation_store_product_wape.png`。
```

```
609
610 ## 5. 关键判断
611
612 1. 严格递推门店-商品粒度下，当前最佳模型为 **{best['model_label']}**，WAPE 为 {
       best['WAPE_pct']:.2f}%。
613 2. 完整综合 Ridge 的 WAPE 为 {full['WAPE_pct']:.2f}%，相对问题一简单指数平滑高 {
       ridge_vs_q1:.2f} 个百分点。因此论文不能写“综合模型在严格 7 日预测中整体优于
       问题一模型”。
614 3. 去天气 Ridge 相比完整 Ridge 的 WAPE 变化为 {weather_gain:.2f} 个百分点。天气
       变量可写作情景补充特征，但不能写成确定因果作用。
615 4. 去活动日 Ridge 相比完整 Ridge 的 WAPE 变化为 {activity_gain:.2f} 个百分点。
       活动日变量可能包含经营安排内生性，论文中应表述为“与活动日标记相关的销量差
       异”。
616 5. 无销量历史 Ridge 相比完整 Ridge 的 WAPE 上升 {history_penalty:.2f} 个百分
       点，说明滞后和滚动销量仍是最核心预测信息。
617 6. 历史日历 Ridge 相比完整 Ridge 的 WAPE 变化为 {calendar_gap:.2f} 个百分点，说
       明外部变量的边际收益需要结合严格递推和未来可得性谨慎解释。
618
619 ## 6. 论文取舍建议
620
621 问题四正文建议以“完整综合 Ridge + 严格递推消融”说明综合模型确实整合了前三问信
       息；但最终模型优劣表述应降调为：综合 Ridge 在门店、商品等聚合层面有较好解释价
       值，在门店-商品最细粒度上未超过问题一简单指数平滑。消融实验适合放在模型评价或
       附录，用来证明团队没有简单堆特征，而是做了可复现的方法取舍。
622 """
623
624
625 def main() -> None:
626     for path in [TABLES, OUTPUTS, FIGURES]:
627         path.mkdir(parents=True, exist_ok=True)
628
629     raw = pd.read_csv(ROOT / "data" / "processed" / "modeling_base_table.csv")
630     raw["date"] = pd.to_datetime(raw["date"])
631     panel = build_store_product_panel(raw, TARGET)
632     panel["date"] = pd.to_datetime(panel["date"])
633     feature_df = prepare_model_frame(panel)
634
635     windows, excluded = validation_windows(feature_df)
636     if not windows:
637         raise RuntimeError("没有可用于消融实验的完整 7 日验证窗口。")
```

```
638
639     predictions = [validate_exp_smoothing(panel, feature_df, windows)]
640     predictions.extend(validate_ridge_spec(panel, feature_df, windows, spec)
            for spec in SPECS)
641     preds = pd.concat(predictions, ignore_index=True)
642     preds["abs_error"] = (preds["actual"] - preds["prediction"]).abs()
643     save_csv(preds, "q4_ablation_predictions.csv")
644
645     metrics = metric_table(preds, windows)
646     save_csv(metrics, "q4_ablation_metrics.csv")
647     plot_store_product_wape(metrics)
648
649     report = build_report(windows, excluded, metrics)
650     (OUTPUTS / "q4_ablation_report.md").write_text(report, encoding="utf-8")
651
652     sp = metrics[metrics["level"] == "store_product"].set_index("model")
653     best = metrics[metrics["level"] == "store_product"].sort_values("WAPE").
            iloc[0]
654     row = (
655         f"| {date.today().isoformat()} | 问题四严格 7 日递推消融实验 | "
656         "processed: modeling_base_table.csv; 未修改 data/raw | "
657         "问题一指数平滑、完整Ridge、去天气/去活动/去事件/历史日历/无历史等 Ridge
                消融 | "
658         "lag/rolling、门店、商品、类别、日历、节假日、活动日、天气按消融组合控制 |
                "
659         f"门店-商品 WAPE: 最佳={best['model_label']} {best['WAPE_pct']:.2f}%; "
660         f"完整Ridge={sp.loc['ridge_full_external','WAPE_pct']:.2f}%; "
661         f"问题一指数平滑={sp.loc['q1_store_product_exp_smoothing','WAPE_pct']:.2
                f}%; "
662         f"去天气={sp.loc['ridge_no_weather','WAPE_pct']:.2f}% | "
663         "历史销量是核心信息; 完整Ridge可作为综合解释模型, 但严格递推细粒度未超过问
                题一指数平滑 | "
664         "消融使用同一验证窗口, 仍不可把天气/活动日关联写成因果; 不据此强行替换最终
                预测文件 | "
665         "将消融表和 WAPE 图写入模型评价或附录 |"
666     )
667     prepend_result_log(row)
668
669     summary = {
670         "validation_mode": "strict_recursive_7day",
```

```
        "windows": [
            {"window_start": str(start.date()), "window_end": str(end.date())}
            for start, end in windows
        ],
        "store_product_metrics": metrics[metrics["level"] == "store_product"][
            ["model", "model_label", "MAE", "RMSE", "WAPE", "WAPE_pct"]
        ].to_dict("records"),
        "best_store_product_model": str(best["model"]),
        "best_store_product_wape_pct": float(best["WAPE_pct"]),
    }
    (OUTPUTS / "q4_ablation_summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(json.dumps(summary, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

Listing 2: 低销量鲁棒性策略检验代码

```
from __future__ import annotations

import json
import sys
from datetime import date
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.evaluation import calculate_metrics
from src.features import build_store_product_panel

TABLES = ROOT / "tables"
OUTPUTS = ROOT / "outputs"
```

```
FIGURES = ROOT / "figures"

TARGET = "positive_sales"
BASE_PREDICTION_PATH = TABLES / "q4_ablation_predictions.csv"


def df_to_md(df: pd.DataFrame, max_rows: int | None = None, digits: int = 3)
    -> str:
    if df is None or df.empty:
        return "无。"
    out = df.copy()
    if max_rows is not None:
        out = out.head(max_rows)
    for col in out.columns:
        if pd.api.types.is_float_dtype(out[col]):
            out[col] = out[col].map(lambda x: "" if pd.isna(x) else f"{x:.{
                digits}f}")
    cols = list(out.columns)
    lines = [
        "| " + " | ".join(cols) + " |",
        "| " + " | ".join(["---"] * len(cols)) + " |",
    ]
    for _, row in out.iterrows():
        lines.append(
            "| "
            + " | ".join("" if pd.isna(row[col]) else str(row[col]) for col in
                cols)
            + " |"
        )
    return "\n".join(lines)


def prepend_result_log(row: str) -> None:
    path = ROOT / "RESULT_LOG.md"
    text = path.read_text(encoding="utf-8")
    if row in text:
        return
    lines = text.splitlines()
    insert_idx = None
    for idx, line in enumerate(lines):
```

```
        if line.startswith("|------"):
            insert_idx = idx + 1
            break
    if insert_idx is not None:
        lines.insert(insert_idx, row)
        path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def save_csv(df: pd.DataFrame, name: str) -> None:
    df.to_csv(TABLES / name, index=False, encoding="utf-8-sig")
    df.to_csv(OUTPUTS / name, index=False, encoding="utf-8-sig")


def demand_class_from_history(values: pd.Series) -> str:
    values = pd.Series(values, dtype=float).fillna(0.0)
    if values.empty or (values > 0).sum() == 0:
        return "no_positive_history"
    avg = float(values.mean())
    zero_ratio = float((values <= 0).mean())
    nonzero_ratio = float((values > 0).mean())
    if nonzero_ratio <= 0.15 or avg < 0.20:
        return "very_sparse"
    if zero_ratio >= 0.75 or avg < 0.50:
        return "low_volume_intermittent"
    if zero_ratio >= 0.50:
        return "intermittent"
    if values.std(ddof=1) / avg >= 1.50:
        return "volatile"
    return "regular"


def is_low_volume_class(label: str) -> bool:
    return label in {"no_positive_history", "very_sparse", "
        low_volume_intermittent"}


def classification_at_window(panel: pd.DataFrame, window_start: pd.Timestamp)
    -> pd.DataFrame:
    history = panel[panel["date"] < window_start].copy()
    combo_cols = ["store_id", "store_name", "product_id", "product_name", "
```

```
        category"]
    combos = (
        panel[combo_cols]
        .drop_duplicates()
        .sort_values(["store_id", "product_id"])
        .reset_index(drop=True)
    )

    product_mean = history.groupby("product_id")[TARGET].mean().to_dict()
    category_mean = history.groupby("category")[TARGET].mean().to_dict()
    global_mean = float(history[TARGET].mean()) if not history.empty else 0.0

    rows = []
    for row in combos.itertuples():
        hist = history[
            (history["store_id"] == row.store_id)
            & (history["product_id"] == row.product_id)
        ].sort_values("date")
        values = hist[TARGET].astype(float)
        recent28 = values.tail(28)
        series_mean = float(values.mean()) if not values.empty else 0.0
        recent28_mean = float(recent28.mean()) if not recent28.empty else
            series_mean
        prod_mean = float(product_mean.get(row.product_id, global_mean))
        cat_mean = float(category_mean.get(row.category, global_mean))
        hierarchy_target = max(
            0.0,
            0.5 * recent28_mean + 0.3 * prod_mean + 0.2 * cat_mean,
        )
        label = demand_class_from_history(values)
        rows.append(
            {
                "window_start": pd.Timestamp(window_start),
                "store_id": int(row.store_id),
                "store_name": row.store_name,
                "product_id": int(row.product_id),
                "product_name": row.product_name,
                "category": row.category,
                "history_days": int(len(values)),
                "history_total_sales": float(values.sum()),
```

```
                "history_average_sales": series_mean,
                "history_zero_sales_ratio": float((values <= 0).mean()) if len(
                    values) else 1.0,
                "history_nonzero_sales_ratio": float((values > 0).mean()) if len
                    (values) else 0.0,
                "recent28_mean": recent28_mean,
                "product_history_mean": prod_mean,
                "category_history_mean": cat_mean,
                "hierarchy_shrink_target": hierarchy_target,
                "demand_class": label,
                "is_low_volume": int(is_low_volume_class(label)),
            }
        )
    return pd.DataFrame(rows)


def build_classification(panel: pd.DataFrame, window_starts: list[pd.Timestamp
    ]) -> pd.DataFrame:
    return pd.concat(
        [classification_at_window(panel, start) for start in window_starts],
        ignore_index=True,
    )


def read_base_predictions() -> pd.DataFrame:
    if not BASE_PREDICTION_PATH.exists():
        raise FileNotFoundError(
            "缺少 q4_ablation_predictions.csv，请先运行 src/stage5_q4_ablation.
                py。"
        )
    preds = pd.read_csv(BASE_PREDICTION_PATH)
    preds["date"] = pd.to_datetime(preds["date"])
    preds["window_start"] = pd.to_datetime(preds["window_start"])
    required_models = {"q1_store_product_exp_smoothing", "ridge_full_external"
        }
    existing = set(preds["model"].unique())
    missing = sorted(required_models - existing)
    if missing:
        raise RuntimeError(f"q4_ablation_predictions.csv 缺少模型：{missing}")
    return preds
```

```
168
169
170 def pivot_base_predictions(preds: pd.DataFrame) -> pd.DataFrame:
171     key_cols = [
172         "validation_mode",
173         "window_start",
174         "window_end",
175         "date",
176         "horizon",
177         "store_id",
178         "store_name",
179         "product_id",
180         "product_name",
181         "category",
182         "actual",
183     ]
184     subset = preds[preds["model"].isin(["q1_store_product_exp_smoothing", "
        ridge_full_external"])]
185     pivot = (
186         subset.pivot_table(
187             index=key_cols,
188             columns="model",
189             values="prediction",
190             aggfunc="first",
191         )
192         .reset_index()
193         .rename_axis(None, axis=1)
194     )
195     return pivot.rename(
196         columns={
197             "q1_store_product_exp_smoothing": "pred_q1_exp_smoothing",
198             "ridge_full_external": "pred_ridge_full",
199         }
200     )
201
202
203 def build_strategy_predictions(base: pd.DataFrame, cls: pd.DataFrame) -> pd.
    DataFrame:
204     merged = base.merge(
205         cls[
```

```
            [
                "window_start",
                "store_id",
                "product_id",
                "history_average_sales",
                "history_zero_sales_ratio",
                "recent28_mean",
                "hierarchy_shrink_target",
                "demand_class",
                "is_low_volume",
            ]
        ],
        on=["window_start", "store_id", "product_id"],
        how="left",
    )
    merged["is_low_volume"] = merged["is_low_volume"].fillna(0).astype(int)
    merged["hierarchy_shrink_target"] = merged["hierarchy_shrink_target"].
        fillna(0.0)
    merged["q1_low_volume_shrunk"] = np.where(
        merged["is_low_volume"].eq(1),
        0.5 * merged["pred_q1_exp_smoothing"] + 0.5 * merged["
            hierarchy_shrink_target"],
        merged["pred_q1_exp_smoothing"],
    )
    merged["ridge_low_volume_shrunk"] = np.where(
        merged["is_low_volume"].eq(1),
        0.5 * merged["pred_ridge_full"] + 0.5 * merged["hierarchy_shrink_target
            "],
        merged["pred_ridge_full"],
    )
    merged["hybrid_low_q1_regular_ridge"] = np.where(
        merged["is_low_volume"].eq(1),
        merged["pred_q1_exp_smoothing"],
        merged["pred_ridge_full"],
    )
    merged["hybrid_low_shrunk_regular_ridge"] = np.where(
        merged["is_low_volume"].eq(1),
        merged["q1_low_volume_shrunk"],
        merged["pred_ridge_full"],
    )
```

```
243
244     strategy_specs = [
245         (
246             "q1_store_product_exp_smoothing",
247             "问题一简单指数平滑",
248             "pred_q1_exp_smoothing",
249             "原问题一门店-商品指数平滑，作为严格递推强基准",
250         ),
251         (
252             "ridge_full_external",
253             "完整综合Ridge",
254             "pred_ridge_full",
255             "原问题四完整综合 Ridge，作为综合模型基准",
256         ),
257         (
258             "q1_low_volume_shrunk",
259             "低销量收缩指数平滑",
260             "q1_low_volume_shrunk",
261             "低销量序列用 50% 指数平滑 + 50% 历史层级均值收缩，其余保持指数平滑",
262         ),
263         (
264             "ridge_low_volume_shrunk",
265             "低销量收缩Ridge",
266             "ridge_low_volume_shrunk",
267             "低销量序列用 50% Ridge + 50% 历史层级均值收缩，其余保持 Ridge",
268         ),
269         (
270             "hybrid_low_q1_regular_ridge",
271             "低销量指数平滑-常规Ridge混合",
272             "hybrid_low_q1_regular_ridge",
273             "低销量序列使用指数平滑，非低销量序列使用完整 Ridge",
274         ),
275         (
276             "hybrid_low_shrunk_regular_ridge",
277             "低销量收缩-常规Ridge混合",
278             "hybrid_low_shrunk_regular_ridge",
279             "低销量序列使用收缩指数平滑，非低销量序列使用完整 Ridge",
280         ),
281     ]
282     rows = []
```

```
    base_cols = [
        "validation_mode",
        "window_start",
        "window_end",
        "date",
        "horizon",
        "store_id",
        "store_name",
        "product_id",
        "product_name",
        "category",
        "actual",
        "history_average_sales",
        "history_zero_sales_ratio",
        "recent28_mean",
        "hierarchy_shrink_target",
        "demand_class",
        "is_low_volume",
    ]
    for model, label, col, rule in strategy_specs:
        tmp = merged[base_cols].copy()
        tmp["prediction"] = np.maximum(0.0, merged[col].astype(float))
        tmp["model"] = model
        tmp["model_label"] = label
        tmp["strategy_rule"] = rule
        rows.append(tmp)
    out = pd.concat(rows, ignore_index=True)
    out["abs_error"] = (out["actual"] - out["prediction"]).abs()
    return out


def aggregate_predictions(preds: pd.DataFrame, level: str) -> pd.DataFrame:
    if level == "store_product":
        return preds.copy()
    if level == "store":
        group_cols = ["date", "window_start", "model", "model_label", "store_id
            ", "store_name"]
    elif level == "product":
        group_cols = [
            "date",
```

```
            "window_start",
            "model",
            "model_label",
            "product_id",
            "product_name",
            "category",
        ]
    elif level == "category":
        group_cols = ["date", "window_start", "model", "model_label", "category
            "]
    else:
        raise ValueError(level)
    return (
        preds.groupby(group_cols, dropna=False, as_index=False)
        .agg(actual=("actual", "sum"), prediction=("prediction", "sum"))
        .sort_values(group_cols)
    )


def metric_table(preds: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for level in ["store_product", "store", "product", "category"]:
        metric_input = aggregate_predictions(preds, level)
        for (model, label), group in metric_input.groupby(["model", "
            model_label"], dropna=False):
            metrics = calculate_metrics(group["actual"], group["prediction"])
            rule = preds.loc[preds["model"] == model, "strategy_rule"].dropna()
                .iloc[0]
            rows.append(
                {
                    "validation_mode": "strict_recursive_7day",
                    "level": level,
                    "model": model,
                    "model_label": label,
                    "strategy_rule": rule,
                    **metrics,
                    "WAPE_pct": metrics["WAPE"] * 100,
                    "actual_sum": float(group["actual"].sum()),
                    "prediction_sum": float(group["prediction"].sum()),
                    "bias": float(group["prediction"].sum() - group["actual"].sum
```

```
                    ()),
                "bias_pct_of_actual": float(
                    (group["prediction"].sum() - group["actual"].sum())
                    / group["actual"].sum()
                )
                if group["actual"].sum() != 0
                else np.nan,
                "n": int(len(group)),
            }
        )
    out = pd.DataFrame(rows)
    sp = out[out["level"] == "store_product"].set_index("model")
    q1_wape = float(sp.loc["q1_store_product_exp_smoothing", "WAPE_pct"])
    ridge_wape = float(sp.loc["ridge_full_external", "WAPE_pct"])
    out["WAPE_pct_point_change_vs_q1"] = out["WAPE_pct"] - q1_wape
    out["WAPE_pct_point_change_vs_ridge"] = out["WAPE_pct"] - ridge_wape
    return out.sort_values(["level", "WAPE", "MAE", "RMSE"]).reset_index(drop=
        True)


def metrics_by_demand_class(preds: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for (model, label, cls), group in preds.groupby(
        ["model", "model_label", "demand_class"], dropna=False
    ):
        metrics = calculate_metrics(group["actual"], group["prediction"])
        rows.append(
            {
                "validation_mode": "strict_recursive_7day",
                "model": model,
                "model_label": label,
                "demand_class": cls,
                "is_low_volume": int(group["is_low_volume"].max()),
                **metrics,
                "WAPE_pct": metrics["WAPE"] * 100,
                "actual_sum": float(group["actual"].sum()),
                "prediction_sum": float(group["prediction"].sum()),
                "n": int(len(group)),
                "series_count": int(
                    group[["window_start", "store_id", "product_id"]].
```

```
                    drop_duplicates().shape[0]
            ),
        }
    )
    return pd.DataFrame(rows).sort_values(["demand_class", "WAPE", "model"]).
        reset_index(drop=True)


def classification_summary(cls: pd.DataFrame) -> pd.DataFrame:
    out = (
        cls.groupby(["window_start", "demand_class", "is_low_volume"], as_index
            =False)
        .agg(
            series_count=("product_id", "count"),
            mean_history_sales=("history_average_sales", "mean"),
            mean_zero_ratio=("history_zero_sales_ratio", "mean"),
        )
        .sort_values(["window_start", "is_low_volume", "demand_class"],
            ascending=[True, False, True])
    )
    out["window_start"] = out["window_start"].dt.date.astype(str)
    return out


def plot_strategy_wape(metrics: pd.DataFrame) -> None:
    sp = metrics[metrics["level"] == "store_product"].sort_values("WAPE_pct")
    label_map = {
        "q1_store_product_exp_smoothing": "Q1 exp smoothing",
        "ridge_full_external": "Full Ridge",
        "q1_low_volume_shrunk": "Low-volume shrink SES",
        "ridge_low_volume_shrunk": "Low-volume shrink Ridge",
        "hybrid_low_q1_regular_ridge": "Low SES + regular Ridge",
        "hybrid_low_shrunk_regular_ridge": "Low shrink + regular Ridge",
    }
    labels = [label_map.get(model, model) for model in sp["model"]]
    values = sp["WAPE_pct"].tolist()
    plt.figure(figsize=(10, 5.4))
    colors = ["#2F6B55" if model == sp.iloc[0]["model"] else "#6B7280" for
        model in sp["model"]]
    plt.barh(labels, values, color=colors)
```

```
    plt.xlabel("WAPE (%)")
    plt.title("Strict recursive 7-day low-volume strategy comparison")
    for idx, value in enumerate(values):
        plt.text(value + 0.2, idx, f"{value:.2f}%", va="center", fontsize=9)
    plt.tight_layout()
    plt.savefig(FIGURES / "q4_low_volume_strategy_wape.png", dpi=180)
    plt.close()


def build_report(
    metrics: pd.DataFrame,
    by_class: pd.DataFrame,
    cls_summary: pd.DataFrame,
) -> str:
    sp = metrics[metrics["level"] == "store_product"].copy()
    sp_table = sp[
        [
            "model_label",
            "strategy_rule",
            "MAE",
            "RMSE",
            "WAPE_pct",
            "WAPE_pct_point_change_vs_q1",
            "WAPE_pct_point_change_vs_ridge",
            "actual_sum",
            "prediction_sum",
            "bias_pct_of_actual",
            "n",
        ]
    ]
    best = sp.sort_values(["WAPE", "MAE", "RMSE"]).iloc[0]
    q1 = sp[sp["model"] == "q1_store_product_exp_smoothing"].iloc[0]
    ridge = sp[sp["model"] == "ridge_full_external"].iloc[0]

    low_volume_rows = by_class[by_class["is_low_volume"] == 1][
        ["model_label", "demand_class", "MAE", "RMSE", "WAPE_pct", "actual_sum"
            , "prediction_sum", "n"]
    ]
    class_counts = (
        cls_summary.groupby(["demand_class", "is_low_volume"], as_index=False)
```

```
        .agg(
            mean_series_count=("series_count", "mean"),
            mean_history_sales=("mean_history_sales", "mean"),
            mean_zero_ratio=("mean_zero_ratio", "mean"),
        )
        .sort_values(["is_low_volume", "demand_class"], ascending=[False, True
            ])
    )
    best_note = (
        "可作为论文中的稳健性补充方案，但仍需说明其只是规则化后处理。"
        if best["model"] not in {"q1_store_product_exp_smoothing", "
            ridge_full_external"}
        else "说明新增低销量后处理没有带来稳定主口径改进，不应强行替换主模型。"
    )

    return f"""# 问题四低销量序列鲁棒性策略检验报告

执行日期：{date.today().isoformat()}

## 1. 检验目的

门店-商品序列中低销量和零销量较多，最细粒度 WAPE 偏高。为避免盲目引入 Croston/TSB
    等复杂间歇需求模型，本检验只评估可解释的低销量分层后处理策略：每个验证窗口开
    始日前，根据历史平均销量、零销量比例和非零销量比例识别低销量序列；低销量序列可
    向“近 28 日均值、商品历史均值、类别历史均值”的层级均值收缩。

分类和收缩目标均只使用窗口开始日前历史数据，未使用验证窗口真实销量。

## 2. 低销量分类概况

{df_to_md(class_counts, 20)}

逐窗口分类明细见 `tables/q4_low_volume_classification.csv`。

## 3. 门店-商品主粒度策略比较

{df_to_md(sp_table, 20)}

完整分层指标见 `tables/q4_low_volume_strategy_metrics.csv`，按需求类别拆分指标
    见 `tables/q4_low_volume_strategy_by_class.csv`。图表已保存至 `figures/
```

```
    q4_low_volume_strategy_wape.png`。
505
506 ## 4. 低销量类别误差
507
508 {df_to_md(low_volume_rows, 30)}
509
510 ## 5. 方法取舍
511
512 严格递推门店-商品粒度下，本检验最佳策略为 **{best['model_label']}**，WAPE={best
    ['WAPE_pct']:.2f}%。问题一简单指数平滑 WAPE={q1['WAPE_pct']:.2f}%，完整综合
    Ridge WAPE={ridge['WAPE_pct']:.2f}%。
513
514 {best_note}
515
516 论文建议把低销量分析写入模型评价和局限性部分：低销量序列是细粒度误差的主要来源之
    一；简单的层级收缩可作为稳健性检查，但若未显著优于主模型，就不应为了“看起来高
    级”而替换最终预测模型。
517 """
518
519
520 def main() -> None:
521     for path in [TABLES, OUTPUTS, FIGURES]:
522         path.mkdir(parents=True, exist_ok=True)
523
524     raw = pd.read_csv(ROOT / "data" / "processed" / "modeling_base_table.csv")
525     raw["date"] = pd.to_datetime(raw["date"])
526     panel = build_store_product_panel(raw, TARGET)
527     panel["date"] = pd.to_datetime(panel["date"])
528
529     base_preds = read_base_predictions()
530     base = pivot_base_predictions(base_preds)
531     window_starts = sorted(pd.to_datetime(base["window_start"]).
        drop_duplicates().tolist())
532     cls = build_classification(panel, window_starts)
533     save_csv(cls, "q4_low_volume_classification.csv")
534
535     strategy_preds = build_strategy_predictions(base, cls)
536     save_csv(strategy_preds, "q4_low_volume_strategy_predictions.csv")
537
538     metrics = metric_table(strategy_preds)
```

```
    by_class = metrics_by_demand_class(strategy_preds)
    cls_summary = classification_summary(cls)
    save_csv(metrics, "q4_low_volume_strategy_metrics.csv")
    save_csv(by_class, "q4_low_volume_strategy_by_class.csv")
    save_csv(cls_summary, "q4_low_volume_classification_summary.csv")
    plot_strategy_wape(metrics)

    report = build_report(metrics, by_class, cls_summary)
    (OUTPUTS / "q4_low_volume_strategy_report.md").write_text(report, encoding
        ="utf-8")

    sp = metrics[metrics["level"] == "store_product"].set_index("model")
    best = metrics[metrics["level"] == "store_product"].sort_values("WAPE").
        iloc[0]
    low_share = float(cls["is_low_volume"].mean())
    row = (
        f"| {date.today().isoformat()} | 问题四低销量序列鲁棒性策略检验 | "
        "processed: modeling_base_table.csv; 输入 q4_ablation_predictions.csv;
            未修改 data/raw | "
        "问题一指数平滑、完整Ridge、低销量层级收缩、低销量/常规混合策略 | "
        "窗口开始日前历史均值、零销量比例、近28日均值、商品均值、类别均值；验证窗
            口内基模型仍为严格递推预测 | "
        f"低销量序列窗口占比={low_share:.1%}；门店-商品 WAPE: 最佳={best['
            model_label']} {best['WAPE_pct']:.2f}%; "
        f"问题一指数平滑={sp.loc['q1_store_product_exp_smoothing','WAPE_pct']:.2
            f}%; "
        f"完整Ridge={sp.loc['ridge_full_external','WAPE_pct']:.2f}% | "
        "低销量策略用于解释和稳健性检查；是否替换主模型以严格递推指标为准 | "
        "分类与收缩只使用窗口开始日前历史数据；层级收缩为规则后处理，不等同于新因
            果机制 | "
        "将低销量误差来源和策略取舍写入模型评价/附录 |"
    )
    prepend_result_log(row)

    summary = {
        "validation_mode": "strict_recursive_7day",
        "low_volume_window_share": low_share,
        "store_product_metrics": metrics[metrics["level"] == "store_product"][
            ["model", "model_label", "MAE", "RMSE", "WAPE", "WAPE_pct"]
        ].to_dict("records"),
```

```
        "best_store_product_model": str(best["model"]),
        "best_store_product_wape_pct": float(best["WAPE_pct"]),
    }
    (OUTPUTS / "q4_low_volume_strategy_summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(json.dumps(summary, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

Listing 3: 低销量混合策略最终预测代码

```
from __future__ import annotations

import json
import sys
from datetime import date
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.features import build_store_product_panel
from src.models import exp_smoothing_prediction, optimize_exp_smoothing_alpha
from src.stage5_q4_low_volume_strategy import classification_at_window

TABLES = ROOT / "tables"
OUTPUTS = ROOT / "outputs"

TARGET = "positive_sales"
FUTURE_START = pd.Timestamp("2022-04-01")


def df_to_md(df: pd.DataFrame, max_rows: int | None = None, digits: int = 3)
    -> str:
```

```
    if df is None or df.empty:
        return "无。"
    out = df.copy()
    if max_rows is not None:
        out = out.head(max_rows)
    for col in out.columns:
        if pd.api.types.is_float_dtype(out[col]):
            out[col] = out[col].map(lambda x: "" if pd.isna(x) else f"{x:.{
                digits}f}")
    cols = list(out.columns)
    lines = [
        "| " + " | ".join(cols) + " |",
        "| " + " | ".join(["---"] * len(cols)) + " |",
    ]
    for _, row in out.iterrows():
        lines.append(
            "| "
            + " | ".join("" if pd.isna(row[col]) else str(row[col]) for col in
                cols)
            + " |"
        )
    return "\n".join(lines)


def prepend_result_log(row: str) -> None:
    path = ROOT / "RESULT_LOG.md"
    text = path.read_text(encoding="utf-8")
    if row in text:
        return
    lines = text.splitlines()
    insert_idx = None
    for idx, line in enumerate(lines):
        if line.startswith("|------"):
            insert_idx = idx + 1
            break
    if insert_idx is not None:
        lines.insert(insert_idx, row)
        path.write_text("\n".join(lines) + "\n", encoding="utf-8")


```

```python
def save_csv(df: pd.DataFrame, name: str) -> None:
    df.to_csv(TABLES / name, index=False, encoding="utf-8-sig")
    df.to_csv(OUTPUTS / name, index=False, encoding="utf-8-sig")


def build_q1_future_predictions(panel: pd.DataFrame, future_template: pd.
    DataFrame) -> pd.DataFrame:
    combo_cols = ["store_id", "store_name", "product_id", "product_name", "
        category"]
    combos = (
        panel[combo_cols]
        .drop_duplicates()
        .sort_values(["store_id", "product_id"])
        .reset_index(drop=True)
    )
    history = {}
    alphas = {}
    for row in combos.itertuples():
        key = (int(row.store_id), int(row.product_id))
        values = (
            panel[
                (panel["store_id"] == row.store_id)
                & (panel["product_id"] == row.product_id)
            ]
            .sort_values("date")[TARGET]
            .astype(float)
            .tolist()
        )
        history[key] = values
        alphas[key] = optimize_exp_smoothing_alpha(pd.Series(values, dtype=
            float))

    ext_cols = [
        "date",
        "weather",
        "is_holiday",
        "is_weekend",
        "is_activity_day",
        "external_scenario_note",
    ]
```

```
    future_ext = future_template[ext_cols].drop_duplicates("date").sort_values("date")
    rows = []
    for ext in future_ext.itertuples():
        for combo in combos.itertuples():
            key = (int(combo.store_id), int(combo.product_id))
            pred = max(0.0, float(exp_smoothing_prediction(history[key], alphas[key])))
            rows.append(
                {
                    "date": pd.Timestamp(ext.date),
                    "store_id": int(combo.store_id),
                    "store_name": combo.store_name,
                    "product_id": int(combo.product_id),
                    "product_name": combo.product_name,
                    "category": combo.category,
                    "weather": ext.weather,
                    "is_holiday": int(ext.is_holiday),
                    "is_weekend": int(ext.is_weekend),
                    "is_activity_day": int(ext.is_activity_day),
                    "pred_q1_exp_smoothing": pred,
                    "external_scenario_note": ext.external_scenario_note,
                }
            )
            history[key].append(pred)
    return pd.DataFrame(rows)


def integerize_forecast(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    out["predicted_sales"] = np.rint(np.maximum(0.0, out["predicted_sales"].astype(float))).astype(int)
    return out


def write_report(
    future: pd.DataFrame,
    store_total: pd.DataFrame,
    category_total: pd.DataFrame,
    cls: pd.DataFrame,
```

```
) -> None:
    strategy_metrics = pd.read_csv(TABLES / "q4_low_volume_strategy_metrics.
        csv")
    sp = strategy_metrics[strategy_metrics["level"] == "store_product"].
        set_index("model")
    low_count = int(cls["is_low_volume"].sum())
    total_count = int(len(cls))
    report = f"""# 问题四低销量混合策略最终预测说明

执行日期：{date.today().isoformat()}

## 1. 生成目的

原最终预测文件 `outputs/final_7day_forecast.csv` 保留为完整综合 Ridge 条件预测。
    本文件新增一个不覆盖原结果的候选最终预测：`outputs/
    final_7day_forecast_hybrid_low_volume.csv`。该策略来自严格 7 日递推低销量鲁
    棒性检验：低销量门店--商品序列使用问题一简单指数平滑，非低销量序列使用完整综合
     Ridge。

## 2. 验证依据

在 2022-03-01 至 2022-03-28 的 4 个完整严格 7 日递推窗口上，低销量指数平滑--常规
     Ridge 混合策略门店--商品 WAPE 为 {sp.loc['hybrid_low_q1_regular_ridge','
    WAPE_pct']:.2f}%，问题一简单指数平滑为 {sp.loc['
    q1_store_product_exp_smoothing','WAPE_pct']:.2f}%，完整综合 Ridge 为 {sp.
    loc['ridge_full_external','WAPE_pct']:.2f}%。

## 3. 未来预测规则

以 2022-04-01 作为预测窗口起点，只使用 2022-03-31 及以前历史销量判定低销量序列。
    本次共有 {low_count}/{total_count} 条门店--商品组合被判为低销量。低销量组合使
    用简单指数平滑递推预测；其余组合沿用完整综合 Ridge 预测。未来天气、温度、风力
    和活动日仍沿用历史同期情景，因此预测仍是条件预测。

## 4. 预测汇总

未来 7 天混合策略预测总销量为 {future['predicted_sales'].sum():.3f}。

门店汇总：

```

```
{df_to_md(store_total, 20)}

类别汇总:

{df_to_md(category_total, 20)}

## 5. 论文表述边界

该混合策略可以作为最终预测候选，因为它在同一严格递推验证口径下略优于原强基准和完整 Ridge；但它是基于低销量诊断的规则化模型选择，不应被写成复杂机器学习模型带来显著突破。正式论文可表述为“在综合 Ridge 的基础上，对低销量稀疏序列采用简单指数平滑兜底，形成稳健混合预测策略”。
"""
    (OUTPUTS / "q4_hybrid_final_forecast_report.md").write_text(report, encoding="utf-8")


def main() -> None:
    for path in [TABLES, OUTPUTS]:
        path.mkdir(parents=True, exist_ok=True)

    ridge_path = OUTPUTS / "final_7day_forecast.csv"
    if not ridge_path.exists():
        raise FileNotFoundError("缺少 outputs/final_7day_forecast.csv。")
    ridge = pd.read_csv(ridge_path)
    ridge["date"] = pd.to_datetime(ridge["date"])
    ridge = ridge.rename(columns={"predicted_sales": "pred_ridge_full"})

    raw = pd.read_csv(ROOT / "data" / "processed" / "modeling_base_table.csv")
    raw["date"] = pd.to_datetime(raw["date"])
    panel = build_store_product_panel(raw, TARGET)
    panel["date"] = pd.to_datetime(panel["date"])

    q1 = build_q1_future_predictions(panel, ridge)
    cls = classification_at_window(panel, FUTURE_START)
    save_csv(cls, "q4_hybrid_future_low_volume_classification.csv")

    merged = ridge.merge(
        q1[
            [
```

```
            "date",
            "store_id",
            "product_id",
            "pred_q1_exp_smoothing",
        ]
    ],
    on=["date", "store_id", "product_id"],
    how="left",
).merge(
    cls[["store_id", "product_id", "demand_class", "is_low_volume"]],
    on=["store_id", "product_id"],
    how="left",
)
merged["is_low_volume"] = merged["is_low_volume"].fillna(0).astype(int)
merged["predicted_sales"] = np.where(
    merged["is_low_volume"].eq(1),
    merged["pred_q1_exp_smoothing"],
    merged["pred_ridge_full"],
)
merged["predicted_sales"] = np.where(
    np.abs(merged["predicted_sales"].astype(float)) < 1e-6,
    0.0,
    merged["predicted_sales"].astype(float),
)
merged["model"] = "hybrid_low_q1_regular_ridge"
merged["model_label"] = "低销量指数平滑-常规Ridge混合"
merged["external_scenario_note"] = (
    merged["external_scenario_note"]
    + "; 低销量组合使用问题一指数平滑，非低销量组合使用综合Ridge"
)
out_cols = [
    "date",
    "store_id",
    "store_name",
    "product_id",
    "product_name",
    "category",
    "weather",
    "is_holiday",
    "is_weekend",
```

```
        "is_activity_day",
        "predicted_sales",
        "model",
        "model_label",
        "demand_class",
        "is_low_volume",
        "external_scenario_note",
    ]
    final = merged[out_cols].sort_values(["date", "store_id", "product_id"]).
        reset_index(drop=True)
    final["date"] = final["date"].dt.date.astype(str)
    save_csv(final, "final_7day_forecast_hybrid_low_volume.csv")
    save_csv(integerize_forecast(final), "
        final_7day_forecast_hybrid_low_volume_integer.csv")

    store_total = (
        final.groupby(["store_id", "store_name"], as_index=False)["
            predicted_sales"]
        .sum()
        .rename(columns={"predicted_sales": "predicted_7day_sales"})
        .sort_values("predicted_7day_sales", ascending=False)
    )
    category_total = (
        final.groupby("category", as_index=False)["predicted_sales"]
        .sum()
        .rename(columns={"predicted_sales": "predicted_7day_sales"})
        .sort_values("predicted_7day_sales", ascending=False)
    )
    product_total = (
        final.groupby(["product_id", "product_name", "category"], as_index=
            False)["predicted_sales"]
        .sum()
        .rename(columns={"predicted_sales": "predicted_7day_sales"})
        .sort_values("predicted_7day_sales", ascending=False)
    )
    save_csv(store_total, "q4_hybrid_forecast_7day_total_by_store.csv")
    save_csv(category_total, "q4_hybrid_forecast_7day_total_by_category.csv")
    save_csv(product_total, "q4_hybrid_forecast_7day_total_by_product.csv")
    write_report(final, store_total, category_total, cls)

```

```
    integer_total = int(integerize_forecast(final)["predicted_sales"].sum())
    summary = {
        "model": "hybrid_low_q1_regular_ridge",
        "future_rows": int(len(final)),
        "low_volume_series": int(cls["is_low_volume"].sum()),
        "total_series": int(len(cls)),
        "future_total_sales": float(final["predicted_sales"].sum()),
        "future_integer_total_sales": integer_total,
        "original_ridge_total_sales": float(ridge["pred_ridge_full"].sum()),
    }
    (OUTPUTS / "q4_hybrid_final_forecast_summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
    )

    row = (
        f"| {date.today().isoformat()} | 问题四低销量混合策略最终预测 | "
        "processed: modeling_base_table.csv; 输入原 final_7day_forecast.csv; 未覆盖原预测表 | "
        "低销量指数平滑 + 常规序列综合 Ridge 混合 | "
        "2022-04-01 前历史销量判定低销量; 低销量组合用指数平滑, 非低销量组合沿用综合 Ridge; 外部变量仍为历史同期情景 | "
        f"未来7天混合预测总量={summary['future_total_sales']:.3f}; 整数化总量={summary['future_integer_total_sales']}; "
        f"低销量组合={summary['low_volume_series']}/{summary['total_series']} | "
        "新增不覆盖原结果的候选最终预测表; 严格递推验证依据来自 q4_low_volume_strategy_metrics.csv | "
        "混合策略为规则化稳健处理, 不应写成复杂模型显著突破; 未来外部变量仍为条件情景 | "
        "论文问题四和模型评价中说明最终预测表取舍 |"
    )
    prepend_result_log(row)
    print(json.dumps(summary, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

Listing 4: 候选优化与不确定性补充代码

```python
from __future__ import annotations

import json
import sys
from datetime import date
from pathlib import Path
from typing import Callable

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.evaluation import calculate_metrics

TABLES = ROOT / "tables"
OUTPUTS = ROOT / "outputs"
FIGURES = ROOT / "figures"

CURRENT_MODEL = "hybrid_current_q1_regular_ridge"
Q1_MODEL = "q1_store_product_exp_smoothing"
RIDGE_MODEL = "ridge_full_external"
IMPROVEMENT_THRESHOLD_PCT = 0.30


def df_to_md(df: pd.DataFrame, max_rows: int | None = None, digits: int = 3)
    -> str:
    if df is None or df.empty:
        return "无。"
    out = df.copy()
    if max_rows is not None:
        out = out.head(max_rows)
    for col in out.columns:
        if pd.api.types.is_float_dtype(out[col]):
            out[col] = out[col].map(lambda x: "" if pd.isna(x) else f"{x:.{
                digits}f}")
    cols = list(out.columns)
    lines = [
```

```
        "| " + " | ".join(cols) + " |",
        "| " + " | ".join(["---"] * len(cols)) + " |",
    ]
    for _, row in out.iterrows():
        lines.append(
            "| "
            + " | ".join("" if pd.isna(row[col]) else str(row[col]) for col in
                cols)
            + " |"
        )
    return "\n".join(lines)


def prepend_result_log(row: str) -> None:
    path = ROOT / "RESULT_LOG.md"
    text = path.read_text(encoding="utf-8")
    if row in text:
        return
    lines = text.splitlines()
    insert_idx = None
    for idx, line in enumerate(lines):
        if line.startswith("|------"):
            insert_idx = idx + 1
            break
    if insert_idx is not None:
        lines.insert(insert_idx, row)
        path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def save_csv(df: pd.DataFrame, name: str) -> None:
    df.to_csv(TABLES / name, index=False, encoding="utf-8-sig")
    df.to_csv(OUTPUTS / name, index=False, encoding="utf-8-sig")


def setup_plot_style() -> None:
    plt.rcParams["font.sans-serif"] = [
        "Microsoft YaHei",
        "SimHei",
        "Arial Unicode MS",
        "DejaVu Sans",
```

```
    ]
    plt.rcParams["axes.unicode_minus"] = False
    plt.rcParams["figure.dpi"] = 150


def load_strategy_base() -> pd.DataFrame:
    strategy_path = TABLES / "q4_low_volume_strategy_predictions.csv"
    cls_path = TABLES / "q4_low_volume_classification.csv"
    if not strategy_path.exists():
        raise FileNotFoundError("缺少 tables/q4_low_volume_strategy_predictions
            .csv。")
    if not cls_path.exists():
        raise FileNotFoundError("缺少 tables/q4_low_volume_classification.csv。
            ")

    preds = pd.read_csv(strategy_path)
    cls = pd.read_csv(cls_path)
    for frame in [preds, cls]:
        frame["window_start"] = pd.to_datetime(frame["window_start"])
    preds["date"] = pd.to_datetime(preds["date"])

    key_cols = [
        "validation_mode",
        "window_start",
        "window_end",
        "date",
        "horizon",
        "store_id",
        "store_name",
        "product_id",
        "product_name",
        "category",
        "actual",
    ]
    model_map = {
        "q1_store_product_exp_smoothing": "pred_q1_exp_smoothing",
        "ridge_full_external": "pred_ridge_full",
    }
    pivot = (
        preds[preds["model"].isin(model_map)]
```

```
        .pivot_table(index=key_cols, columns="model", values="prediction",
            aggfunc="first")
        .reset_index()
        .rename_axis(None, axis=1)
        .rename(columns=model_map)
    )
    required = {"pred_q1_exp_smoothing", "pred_ridge_full"}
    missing = required - set(pivot.columns)
    if missing:
        raise ValueError(f"低销量策略预测表缺少必要基准预测列: {sorted(missing)}"
            )

    cls_cols = [
        "window_start",
        "store_id",
        "product_id",
        "history_average_sales",
        "history_zero_sales_ratio",
        "recent28_mean",
        "product_history_mean",
        "category_history_mean",
        "hierarchy_shrink_target",
        "demand_class",
        "is_low_volume",
    ]
    out = pivot.merge(cls[cls_cols], on=["window_start", "store_id", "
        product_id"], how="left")
    out["is_low_volume"] = out["is_low_volume"].fillna(0).astype(int)
    numeric_cols = [
        "actual",
        "pred_q1_exp_smoothing",
        "pred_ridge_full",
        "history_average_sales",
        "history_zero_sales_ratio",
        "recent28_mean",
        "product_history_mean",
        "category_history_mean",
        "hierarchy_shrink_target",
    ]
    for col in numeric_cols:
```

```
153         out[col] = pd.to_numeric(out[col], errors="coerce").fillna(0.0)
154     out["window_end"] = pd.to_datetime(out["window_end"])
155     return out.sort_values(["window_start", "date", "store_id", "product_id"])
            .reset_index(
156         drop=True
157     )
158
159
160 def candidate_thresholds() -> list[tuple[str, str, str, Callable[[pd.DataFrame
        ], pd.Series]]]:
161     return [
162         (
163             "current_rule",
164             "当前低销量规则",
165             "沿用 no_positive_history、very_sparse、low_volume_intermittent 三类
                低销量判定",
166             lambda df: df["is_low_volume"].eq(1),
167         ),
168         (
169             "avg_lt_0_2",
170             "历史均值 <0.2",
171             "窗口开始日前门店-商品历史日均销量小于 0.2",
172             lambda df: df["history_average_sales"].lt(0.2),
173         ),
174         (
175             "avg_lt_0_3",
176             "历史均值 <0.3",
177             "窗口开始日前门店-商品历史日均销量小于 0.3",
178             lambda df: df["history_average_sales"].lt(0.3),
179         ),
180         (
181             "zero_ge_0_6",
182             "零销量比例 >=0.6",
183             "窗口开始日前零销量天数占比不低于 60%",
184             lambda df: df["history_zero_sales_ratio"].ge(0.6),
185         ),
186         (
187             "zero_ge_0_75",
188             "零销量比例 >=0.75",
189             "窗口开始日前零销量天数占比不低于 75%",
```

```
            lambda df: df["history_zero_sales_ratio"].ge(0.75),
        ),
        (
            "avg_lt_0_3_or_zero_ge_0_75",
            "历史均值 <0.3 或零销量比例 >=0.75",
            "低均值和高零销量比例的并集判定",
            lambda df: df["history_average_sales"].lt(0.3)
            | df["history_zero_sales_ratio"].ge(0.75),
        ),
    ]


def low_volume_predictors() -> list[tuple[str, str, str, Callable[[pd.
    DataFrame], pd.Series]]]:
    return [
        (
            "q1_exp_smoothing",
            "低销量用指数平滑",
            "低销量序列使用问题一门店-商品指数平滑，常规序列使用完整 Ridge",
            lambda df: df["pred_q1_exp_smoothing"],
        ),
        (
            "recent28_mean",
            "低销量用近28日均值",
            "低销量序列使用窗口开始日前近 28 日均值，常规序列使用完整 Ridge",
            lambda df: df["recent28_mean"],
        ),
        (
            "product_mean",
            "低销量用商品均值",
            "低销量序列使用窗口开始日前同商品历史均值，常规序列使用完整 Ridge",
            lambda df: df["product_history_mean"],
        ),
        (
            "category_mean",
            "低销量用类别均值",
            "低销量序列使用窗口开始日前同类别历史均值，常规序列使用完整 Ridge",
            lambda df: df["category_history_mean"],
        ),
        (
```

```
            "hierarchy_shrink",
            "低销量用层级收缩均值",
            "低销量序列使用近28日、商品均值、类别均值的层级收缩目标，常规序列使用完整 Ridge",
            lambda df: df["hierarchy_shrink_target"],
        ),
        (
            "q1_hierarchy_blend",
            "低销量指数平滑+层级收缩",
            "低销量序列使用 70% 指数平滑 + 30% 层级收缩目标，常规序列使用完整 Ridge",
            lambda df: 0.7 * df["pred_q1_exp_smoothing"]
            + 0.3 * df["hierarchy_shrink_target"],
        ),
    ]


def base_columns() -> list[str]:
    return [
        "validation_mode",
        "window_start",
        "window_end",
        "date",
        "horizon",
        "store_id",
        "store_name",
        "product_id",
        "product_name",
        "category",
        "actual",
        "history_average_sales",
        "history_zero_sales_ratio",
        "demand_class",
        "is_low_volume",
    ]


def build_candidate_predictions(base: pd.DataFrame) -> pd.DataFrame:
    rows = []

```

```
    def append_model(
        model: str,
        label: str,
        rule: str,
        prediction: pd.Series | np.ndarray,
        low_mask: pd.Series | None = None,
        threshold_label: str = "不适用",
    ) -> None:
        tmp = base[base_columns()].copy()
        tmp["prediction"] = np.maximum(0.0, pd.Series(prediction, index=base.
            index).astype(float))
        tmp["model"] = model
        tmp["model_label"] = label
        tmp["strategy_rule"] = rule
        tmp["threshold_rule"] = threshold_label
        tmp["low_volume_selected"] = low_mask.astype(int).to_numpy() if
            low_mask is not None else 0
        rows.append(tmp)

    append_model(
        Q1_MODEL,
        "问题一简单指数平滑",
        "所有门店-商品序列均使用问题一指数平滑",
        base["pred_q1_exp_smoothing"],
    )
    append_model(
        RIDGE_MODEL,
        "完整综合Ridge",
        "所有门店-商品序列均使用完整综合 Ridge",
        base["pred_ridge_full"],
    )

    for threshold_id, threshold_label, threshold_rule, mask_fn in
        candidate_thresholds():
        low_mask = mask_fn(base).fillna(False)
        for pred_id, pred_label, pred_rule, pred_fn in low_volume_predictors():
            pred = np.where(low_mask, pred_fn(base), base["pred_ridge_full"])
            model = f"hybrid_{threshold_id}_{pred_id}"
            label = f"{threshold_label}-{pred_label}"
            if threshold_id == "current_rule" and pred_id == "q1_exp_smoothing"
```

```
                :
                model = CURRENT_MODEL
                label = "当前混合策略: 低销量指数平滑-常规Ridge"
            append_model(
                model=model,
                label=label,
                rule=pred_rule,
                prediction=pred,
                low_mask=low_mask,
                threshold_label=threshold_label,
            )

    candidates = pd.concat(rows, ignore_index=True)
    candidates["abs_error"] = (candidates["actual"] - candidates["prediction"
        ]).abs()
    return candidates


def apply_q1_aggregate_calibration(
    base: pd.DataFrame,
    candidate_preds: pd.DataFrame,
    level: str,
) -> pd.DataFrame:
    group_map = {
        "overall": [],
        "store": ["store_id"],
        "product": ["product_id"],
        "category": ["category"],
    }
    group_cols = group_map[level]
    current = candidate_preds[candidate_preds["model"] == CURRENT_MODEL].copy
        ()
    if current.empty:
        raise ValueError("候选预测中找不到当前混合策略。")
    current = current.drop(columns=["prediction"])
    current = current.merge(
        base[
            [
                "window_start",
                "date",
```

```
            "store_id",
            "product_id",
            "category",
            "pred_q1_exp_smoothing",
        ]
    ],
    on=["window_start", "date", "store_id", "product_id", "category"],
    how="left",
)
current = current.rename(columns={"pred_q1_exp_smoothing": "
    q1_target_prediction"})
original = candidate_preds.loc[
    candidate_preds["model"] == CURRENT_MODEL,
    ["window_start", "date", "store_id", "product_id", "category", "
        prediction"],
]
current = current.merge(
    original,
    on=["window_start", "date", "store_id", "product_id", "category"],
    how="left",
)

sum_cols = ["window_start", "date", *group_cols]
factor = (
    current.groupby(sum_cols, dropna=False, as_index=False)
    .agg(
        pred_sum=("prediction", "sum"),
        q1_sum=("q1_target_prediction", "sum"),
    )
    .reset_index(drop=True)
)
factor["calibration_factor"] = np.where(
    factor["pred_sum"].abs().gt(1e-9),
    factor["q1_sum"] / factor["pred_sum"],
    1.0,
)
# Avoid making a diagnostic calibration dominate the validation through
    extreme ratios.
factor["calibration_factor"] = factor["calibration_factor"].clip(lower
    =0.5, upper=1.5)
```

```
    out = current.merge(factor[sum_cols + ["calibration_factor"]], on=sum_cols, how="left")
    out["prediction"] = np.maximum(0.0, out["prediction"] * out["calibration_factor"].fillna(1.0))
    out["model"] = f"hybrid_current_calibrated_to_q1_{level}"
    level_label = {"overall": "总量", "store": "门店", "product": "商品", "category": "类别"}[level]
    out["model_label"] = f"当前混合策略校准到问题一{level_label}预测"
    out["strategy_rule"] = (
        f"以后处理方式将当前混合策略在日期-{level_label}层面的预测总量按比例缩放到问题一指数平滑聚合预测；"
        "比例只使用两套模型预测值，不使用验证窗口真实销量"
    )
    out["threshold_rule"] = f"问题一{level_label}聚合预测后处理校准"
    return out[candidate_preds.columns]


def add_calibrated_candidates(base: pd.DataFrame, candidates: pd.DataFrame) -> pd.DataFrame:
    calibrated = [
        apply_q1_aggregate_calibration(base, candidates, level)
        for level in ["overall", "store", "product", "category"]
    ]
    return pd.concat([candidates, *calibrated], ignore_index=True)


def aggregate_predictions(preds: pd.DataFrame, level: str) -> pd.DataFrame:
    if level == "store_product":
        return preds.copy()
    if level == "store":
        group_cols = ["date", "window_start", "model", "model_label", "store_id", "store_name"]
    elif level == "product":
        group_cols = [
            "date",
            "window_start",
            "model",
            "model_label",
            "product_id",
            "product_name",
```

```
            "category",
        ]
    elif level == "category":
        group_cols = ["date", "window_start", "model", "model_label", "category
            "]
    else:
        raise ValueError(level)
    return (
        preds.groupby(group_cols, dropna=False, as_index=False)
        .agg(actual=("actual", "sum"), prediction=("prediction", "sum"))
        .sort_values(group_cols)
    )


def metric_row(df: pd.DataFrame) -> dict:
    metrics = calculate_metrics(df["actual"], df["prediction"])
    actual_sum = float(df["actual"].sum())
    prediction_sum = float(df["prediction"].sum())
    return {
        **metrics,
        "WAPE_pct": metrics["WAPE"] * 100 if pd.notna(metrics["WAPE"]) else np.
            nan,
        "actual_sum": actual_sum,
        "prediction_sum": prediction_sum,
        "abs_error_sum": float((df["actual"] - df["prediction"]).abs().sum()),
        "bias": prediction_sum - actual_sum,
        "bias_pct_of_actual": (prediction_sum - actual_sum) / actual_sum
        if actual_sum != 0
        else np.nan,
        "n": int(len(df)),
    }


def build_metric_table(preds: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for level in ["store_product", "store", "product", "category"]:
        agg = aggregate_predictions(preds, level)
        for (model, label), group in agg.groupby(["model", "model_label"],
            dropna=False):
            original = preds[preds["model"] == model]
```

```
            row = {
                "validation_mode": "strict_recursive_7day",
                "level": level,
                "model": model,
                "model_label": label,
                "strategy_rule": original["strategy_rule"].dropna().iloc[0],
                "threshold_rule": original["threshold_rule"].dropna().iloc[0],
                "low_volume_selected_share": float(original["low_volume_selected
                    "].mean()),
                **metric_row(group),
            }
            rows.append(row)
    out = pd.DataFrame(rows)
    sp = out[out["level"] == "store_product"].set_index("model")
    current_wape = float(sp.loc[CURRENT_MODEL, "WAPE_pct"])
    q1_wape = float(sp.loc[Q1_MODEL, "WAPE_pct"])
    ridge_wape = float(sp.loc[RIDGE_MODEL, "WAPE_pct"])
    out["WAPE_pct_point_change_vs_current"] = out["WAPE_pct"] - current_wape
    out["WAPE_pct_point_change_vs_q1"] = out["WAPE_pct"] - q1_wape
    out["WAPE_pct_point_change_vs_ridge"] = out["WAPE_pct"] - ridge_wape
    return out.sort_values(["level", "WAPE", "MAE", "RMSE"]).reset_index(drop=
        True)


def build_window_metric_table(preds: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for (model, label, window_start), group in preds.groupby(
        ["model", "model_label", "window_start"], dropna=False
    ):
        rows.append(
            {
                "validation_mode": "strict_recursive_7day",
                "window_start": pd.Timestamp(window_start).date().isoformat(),
                "model": model,
                "model_label": label,
                **metric_row(group),
            }
        )
    out = pd.DataFrame(rows)
    current = out[out["model"] == CURRENT_MODEL][["window_start", "WAPE_pct"
```

```
        ]].rename(
486         columns={"WAPE_pct": "current_window_WAPE_pct"}
487     )
488     out = out.merge(current, on="window_start", how="left")
489     out["WAPE_pct_point_change_vs_current_window"] = (
490         out["WAPE_pct"] - out["current_window_WAPE_pct"]
491     )
492     return out.sort_values(["window_start", "WAPE", "MAE", "RMSE"]).
        reset_index(drop=True)
493
494
495 def build_leave_one_window_selection(preds: pd.DataFrame) -> pd.DataFrame:
496     # Calibration rows are diagnostics, not competing low-volume decision
        rules.
497     candidate_pool = preds[~preds["model"].str.contains("calibrated", regex=
        False)].copy()
498     windows = sorted(pd.to_datetime(candidate_pool["window_start"]).
        drop_duplicates())
499     rows = []
500     for holdout in windows:
501         train = candidate_pool[candidate_pool["window_start"] != holdout]
502         holdout_df = candidate_pool[candidate_pool["window_start"] == holdout]
503         train_scores = []
504         for (model, label), group in train.groupby(["model", "model_label"],
            dropna=False):
505             train_scores.append({"model": model, "model_label": label, **
                metric_row(group)})
506         selected = pd.DataFrame(train_scores).sort_values(["WAPE", "MAE", "RMSE
            "]).iloc[0]
507         eval_models = [
508             (selected["model"], selected["model_label"], "
                selected_by_other_windows"),
509             (CURRENT_MODEL, "当前混合策略：低销量指数平滑-常规Ridge", "
                current_hybrid"),
510             (Q1_MODEL, "问题一简单指数平滑", "q1_baseline"),
511             (RIDGE_MODEL, "完整综合Ridge", "ridge_baseline"),
512         ]
513         for model, label, eval_type in eval_models:
514             group = holdout_df[holdout_df["model"] == model]
515             rows.append(
```

```
                {
                    "holdout_window_start": holdout.date().isoformat(),
                    "selected_model_on_other_windows": selected["model"],
                    "selected_label_on_other_windows": selected["model_label"],
                    "selected_train_WAPE_pct": float(selected["WAPE_pct"]),
                    "eval_type": eval_type,
                    "eval_model": model,
                    "eval_label": label,
                    **metric_row(group),
                }
            )
    out = pd.DataFrame(rows)
    selected = out[out["eval_type"] == "selected_by_other_windows"][
        ["holdout_window_start", "WAPE_pct"]
    ].rename(columns={"WAPE_pct": "selected_holdout_WAPE_pct"})
    current = out[out["eval_type"] == "current_hybrid"][
        ["holdout_window_start", "WAPE_pct"]
    ].rename(columns={"WAPE_pct": "current_holdout_WAPE_pct"})
    out = out.merge(selected, on="holdout_window_start", how="left").merge(
        current, on="holdout_window_start", how="left"
    )
    out["WAPE_pct_point_change_vs_current_holdout"] = (
        out["WAPE_pct"] - out["current_holdout_WAPE_pct"]
    )
    return out.sort_values(["holdout_window_start", "eval_type"]).reset_index(
        drop=True)


def load_calendar() -> pd.DataFrame:
    path = ROOT / "data" / "processed" / "modeling_base_table.csv"
    calendar = pd.read_csv(
        path,
        usecols=lambda col: col
        in {"date", "weekday", "is_weekend", "is_holiday", "is_activity_day"},
    )
    calendar["date"] = pd.to_datetime(calendar["date"])
    calendar = calendar.drop_duplicates("date")
    for col in ["weekday", "is_weekend", "is_holiday", "is_activity_day"]:
        calendar[col] = pd.to_numeric(calendar[col], errors="coerce").fillna(0)
            .astype(int)
```

```
    return calendar


def grouped_metrics(preds: pd.DataFrame, group_cols: list[str]) -> pd.
    DataFrame:
    rows = []
    for keys, group in preds.groupby(group_cols, dropna=False):
        if not isinstance(keys, tuple):
            keys = (keys,)
        row = dict(zip(group_cols, keys))
        row.update(metric_row(group))
        rows.append(row)
    if not rows:
        return pd.DataFrame()
    return pd.DataFrame(rows).sort_values(group_cols).reset_index(drop=True)


def build_error_attribution(
    candidates: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, dict]:
    hybrid = candidates[candidates["model"] == CURRENT_MODEL].copy()
    hybrid = hybrid.merge(load_calendar(), on="date", how="left")
    by_horizon = grouped_metrics(hybrid, ["horizon"])
    by_class = grouped_metrics(hybrid, ["demand_class", "is_low_volume"])

    segment_rows = []
    segment_defs = [
        ("工作日", "is_weekend=0 且 is_holiday=0", (hybrid["is_weekend"] == 0)
            & (hybrid["is_holiday"] == 0)),
        ("周末", "is_weekend=1", hybrid["is_weekend"] == 1),
        ("非周末", "is_weekend=0", hybrid["is_weekend"] == 0),
        ("活动日", "is_activity_day=1", hybrid["is_activity_day"] == 1),
        ("非活动日", "is_activity_day=0", hybrid["is_activity_day"] == 0),
        ("节假日", "is_holiday=1", hybrid["is_holiday"] == 1),
        ("非节假日", "is_holiday=0", hybrid["is_holiday"] == 0),
    ]
    for label, definition, mask in segment_defs:
        subset = hybrid.loc[mask].copy()
        if subset.empty:
            segment_rows.append(
```

```
                {
                    "calendar_segment": label,
                    "definition": definition,
                    "MAE": np.nan,
                    "RMSE": np.nan,
                    "WAPE": np.nan,
                    "WAPE_pct": np.nan,
                    "actual_sum": 0.0,
                    "prediction_sum": 0.0,
                    "abs_error_sum": 0.0,
                    "bias": 0.0,
                    "bias_pct_of_actual": np.nan,
                    "n": 0,
                    "date_count": 0,
                }
            )
        else:
            row = {"calendar_segment": label, "definition": definition, **
                metric_row(subset)}
            row["date_count"] = int(subset["date"].nunique())
            segment_rows.append(row)
    by_calendar = pd.DataFrame(segment_rows)
    save_csv(by_horizon, "q4_hybrid_error_by_horizon.csv")
    save_csv(by_class, "q4_hybrid_error_by_demand_class.csv")
    save_csv(by_calendar, "q4_hybrid_error_by_calendar_segment.csv")
    low = hybrid[hybrid["is_low_volume"] == 1]
    regular = hybrid[hybrid["is_low_volume"] == 0]
    return by_horizon, by_class, by_calendar, {
        "low_volume": metric_row(low) if not low.empty else {},
        "regular": metric_row(regular) if not regular.empty else {},
    }


def aggregate_7day_window(preds: pd.DataFrame, group_cols: list[str]) -> pd.
    DataFrame:
    return (
        preds.groupby(["window_start", *group_cols], dropna=False, as_index=
            False)
        .agg(
            actual_7day_sales=("actual", "sum"),
```

```
            predicted_7day_sales=("prediction", "sum"),
        )
        .assign(
            residual=lambda df: df["actual_7day_sales"] - df["
                predicted_7day_sales"],
            abs_residual=lambda df: df["residual"].abs(),
        )
    )


def interval_table_for_level(
    validation: pd.DataFrame,
    future: pd.DataFrame,
    level: str,
    group_cols: list[str],
) -> pd.DataFrame:
    val = aggregate_7day_window(validation, group_cols)
    stats = (
        val.groupby(group_cols, dropna=False)
        .agg(
            residual_q10=("residual", lambda x: float(np.quantile(x, 0.10))),
            residual_median=("residual", "median"),
            residual_q90=("residual", lambda x: float(np.quantile(x, 0.90))),
            residual_mean=("residual", "mean"),
            mean_abs_residual=("abs_residual", "mean"),
            residual_std=("residual", "std"),
            residual_count=("residual", "count"),
            validation_actual_7day_mean=("actual_7day_sales", "mean"),
            validation_prediction_7day_mean=("predicted_7day_sales", "mean"),
        )
        .reset_index()
    )
    fut = (
        future.groupby(group_cols, dropna=False, as_index=False)["
            predicted_sales"]
        .sum()
        .rename(columns={"predicted_sales": "point_forecast_7day_sales"})
    )
    out = fut.merge(stats, on=group_cols, how="left")
    out["interval_method"] = "严格递推验证7日残差经验10%-90%分位数"
```

```
    raw_lower = out["point_forecast_7day_sales"] + out["residual_q10"]
    raw_upper = out["point_forecast_7day_sales"] + out["residual_q90"]
    out["lower_80pct_empirical"] = np.maximum(
        0.0, np.minimum(out["point_forecast_7day_sales"], raw_lower)
    )
    out["median_residual_adjusted_forecast"] = np.maximum(
        0.0, out["point_forecast_7day_sales"] + out["residual_median"]
    )
    out["upper_80pct_empirical"] = np.maximum(out["point_forecast_7day_sales"
        ], raw_upper)
    out.insert(0, "level", level)
    return out.sort_values("point_forecast_7day_sales", ascending=False).
        reset_index(drop=True)


def build_forecast_intervals(
    candidates: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    future_path = OUTPUTS / "final_7day_forecast_hybrid_low_volume.csv"
    if not future_path.exists():
        raise FileNotFoundError("缺少 outputs/
            final_7day_forecast_hybrid_low_volume.csv。")
    future = pd.read_csv(future_path)
    future["date"] = pd.to_datetime(future["date"])
    future["predicted_sales"] = pd.to_numeric(future["predicted_sales"],
        errors="coerce").fillna(0.0)

    validation = candidates[candidates["model"] == CURRENT_MODEL].copy()
    store = interval_table_for_level(
        validation,
        future,
        "store",
        ["store_id", "store_name"],
    )
    product = interval_table_for_level(
        validation,
        future,
        "product",
        ["product_id", "product_name", "category"],
    )
```

```
    category = interval_table_for_level(validation, future, "category", ["category"])
    save_csv(store, "final_forecast_interval_by_store.csv")
    save_csv(product, "final_forecast_interval_by_product.csv")
    save_csv(category, "final_forecast_interval_by_category.csv")
    return store, product, category


def plot_outputs(
    metrics: pd.DataFrame,
    by_horizon: pd.DataFrame,
    store_interval: pd.DataFrame,
) -> None:
    setup_plot_style()

    sp = metrics[metrics["level"] == "store_product"].head(12).sort_values("WAPE_pct")
    fig, ax = plt.subplots(figsize=(10.5, 6.0))
    labels = sp["model_label"].str.slice(0, 28)
    colors = ["#2F6B55" if m == CURRENT_MODEL else "#6B7280" for m in sp["model"]]
    ax.barh(labels, sp["WAPE_pct"], color=colors)
    ax.set_xlabel("WAPE (%)")
    ax.set_title("严格递推候选模型 WAPE 前 12")
    for idx, row in enumerate(sp.itertuples()):
        ax.text(row.WAPE_pct + 0.15, idx, f"{row.WAPE_pct:.2f}%", va="center", fontsize=8)
    fig.tight_layout()
    fig.savefig(FIGURES / "q4_candidate_model_wape.png", dpi=180)
    plt.close(fig)

    horizon = by_horizon.sort_values("horizon")
    fig, ax = plt.subplots(figsize=(8.5, 4.8))
    ax.bar(horizon["horizon"].astype(str), horizon["WAPE_pct"], color="#4C78A8")
    ax.set_xlabel("预测步长 horizon")
    ax.set_ylabel("WAPE (%)")
    ax.set_title("当前混合策略按 horizon 的严格递推误差")
    for idx, row in enumerate(horizon.itertuples()):
        ax.text(idx, row.WAPE_pct + 0.6, f"{row.WAPE_pct:.1f}%", ha="center",
```

```
            fontsize=8)
    fig.tight_layout()
    fig.savefig(FIGURES / "q4_hybrid_error_by_horizon.png", dpi=180)
    plt.close(fig)

    store = store_interval.sort_values("point_forecast_7day_sales", ascending=
        True)
    fig, ax = plt.subplots(figsize=(10, 5.5))
    x = store["point_forecast_7day_sales"].to_numpy()
    lower = x - store["lower_80pct_empirical"].to_numpy()
    upper = store["upper_80pct_empirical"].to_numpy() - x
    ax.errorbar(
        x,
        store["store_name"],
        xerr=np.vstack([lower, upper]),
        fmt="o",
        color="#2F6B55",
        ecolor="#9CA3AF",
        capsize=3,
    )
    ax.set_xlabel("未来 7 天预测销量")
    ax.set_title("门店未来 7 天预测及经验残差区间")
    fig.tight_layout()
    fig.savefig(FIGURES / "final_forecast_interval_by_store.png", dpi=180)
    plt.close(fig)

    store_total_path = TABLES / "q4_hybrid_forecast_7day_total_by_store.csv"
    category_total_path = TABLES / "q4_hybrid_forecast_7day_total_by_category.
        csv"
    if store_total_path.exists():
        store_total = pd.read_csv(store_total_path).sort_values(
            "predicted_7day_sales", ascending=True
        )
        fig, ax = plt.subplots(figsize=(10, 5.4))
        ax.barh(store_total["store_name"], store_total["predicted_7day_sales"],
             color="#2F6B55")
        ax.set_xlabel("预测 7 日销量")
        ax.set_title("低销量混合策略门店 7 日预测总量")
        for idx, row in enumerate(store_total.itertuples()):
            ax.text(
```

```
                row.predicted_7day_sales + 3,
                idx,
                f"{row.predicted_7day_sales:.1f}",
                va="center",
                fontsize=8,
            )
        fig.tight_layout()
        fig.savefig(FIGURES / "q4_hybrid_forecast_7day_total_by_store.png", dpi
            =180)
        plt.close(fig)
    if category_total_path.exists():
        category_total = pd.read_csv(category_total_path).sort_values(
            "predicted_7day_sales", ascending=True
        )
        fig, ax = plt.subplots(figsize=(10, 5.4))
        ax.barh(category_total["category"], category_total["
            predicted_7day_sales"], color="#4C78A8")
        ax.set_xlabel("预测 7 日销量")
        ax.set_title("低销量混合策略类别 7 日预测总量")
        for idx, row in enumerate(category_total.itertuples()):
            ax.text(
                row.predicted_7day_sales + 3,
                idx,
                f"{row.predicted_7day_sales:.1f}",
                va="center",
                fontsize=8,
            )
        fig.tight_layout()
        fig.savefig(FIGURES / "q4_hybrid_forecast_7day_total_by_category.png",
            dpi=180)
        plt.close(fig)


def build_candidate_review(
    metrics: pd.DataFrame,
    window_metrics: pd.DataFrame,
    leave_one: pd.DataFrame,
) -> str:
    sp = metrics[metrics["level"] == "store_product"].copy()
    show_cols = [
```

```
        "model_label",
        "threshold_rule",
        "MAE",
        "RMSE",
        "WAPE_pct",
        "WAPE_pct_point_change_vs_current",
        "actual_sum",
        "prediction_sum",
        "bias_pct_of_actual",
        "low_volume_selected_share",
    ]
    top = sp[show_cols].head(15)
    current = sp[sp["model"] == CURRENT_MODEL].iloc[0]
    q1 = sp[sp["model"] == Q1_MODEL].iloc[0]
    ridge = sp[sp["model"] == RIDGE_MODEL].iloc[0]
    best = sp.iloc[0]
    improvement = current["WAPE_pct"] - best["WAPE_pct"]
    selected_mean = float(
        leave_one.loc[leave_one["eval_type"] == "selected_by_other_windows", "
            WAPE_pct"].mean()
    )
    current_holdout_mean = float(
        leave_one.loc[leave_one["eval_type"] == "current_hybrid", "WAPE_pct"].
            mean()
    )
    holdout_improvement = current_holdout_mean - selected_mean
    replace_note = (
        f"全样本最佳候选相对当前改善 {improvement:.3f} 个百分点，但留一窗口选模后
            的平均改善为 "
        f"{holdout_improvement:.3f} 个百分点，低于 0.3 个百分点阈值。因此本轮不自
            动替换冻结主方案，"
        "只将近 28 日均值兜底记为探索性候选。"
    )
    calibration = sp[sp["model"].str.contains("calibrated", regex=False)][
        show_cols]
    best_windows = window_metrics[window_metrics["model"] == best["model"]][
        [
            "window_start",
            "model_label",
            "WAPE_pct",
```

```
            "WAPE_pct_point_change_vs_current_window",
            "actual_sum",
            "prediction_sum",
        ]
    ]
    leave_cols = [
        "holdout_window_start",
        "selected_label_on_other_windows",
        "eval_type",
        "eval_label",
        "WAPE_pct",
        "WAPE_pct_point_change_vs_current_holdout",
    ]

    return f"""# 问题四候选模型有限优化复核

执行日期: {date.today().isoformat()}

## 1. 复核目的

本轮只围绕当前严格 7 日递推口径做有限优化，重点检查两类问题：第一，低销量判定阈值和低销量兜底预测方法是否能稳定优于当前混合策略；第二，门店、商品、类别层级的比例校准是否能改善主粒度 WAPE。所有候选均基于已经生成的严格递推验证明细，不使用验证窗口真实销量构造未来特征，也不覆盖原最终预测表。

## 2. 主粒度候选结果

{df_to_md(top, 20)}

完整结果见 `tables/q4_candidate_model_metrics.csv`，候选预测明细见 `tables/q4_candidate_model_predictions.csv`，对比图见 `figures/q4_candidate_model_wape.png`。

## 3. 当前基准与替换判断

- 当前混合策略 WAPE={current['WAPE_pct']:.3f}%。
- 问题一简单指数平滑 WAPE={q1['WAPE_pct']:.3f}%。
- 完整综合 Ridge WAPE={ridge['WAPE_pct']:.3f}%。
- 本轮最佳候选为 `{best['model_label']}`，WAPE={best['WAPE_pct']:.3f}%，相对当前混合策略变化 {best['WAPE_pct_point_change_vs_current']:.3f} 个百分点。
```

```
880
881 结论：{replace_note}
882
883 ## 4. 窗口级稳健性
884
885 全样本最佳候选在各验证窗口的表现如下。负数表示该窗口优于当前混合策略：
886
887 {df_to_md(best_windows, 10)}
888
889 进一步用“留一窗口”方式检查模型选择偏差：每次用另外 3 个窗口选择候选模型，再在留
        出的 1 个窗口上评价。结果如下：
890
891 {df_to_md(leave_one[leave_cols], 20)}
892
893 留一窗口下，选中候选的平均 WAPE={selected_mean:.3f}%，当前混合策略平均 WAPE={
        current_holdout_mean:.3f}%，平均改善 {holdout_improvement:.3f} 个百分点。该
        提升不足以把探索性候选写成新的主方案。
894
895 ## 5. 层级校准检查
896
897 层级校准只把当前混合策略的日期--层级预测总量按比例缩放到问题一指数平滑的同层级聚合
        预测；该操作不使用验证期真实销量，因此属于可复现后处理，而不是利用真实答案校
        准。结果如下：
898
899 {df_to_md(calibration, 10)}
900
901 若层级校准不能在门店--商品主粒度上明显改善 WAPE，则论文中只保留“检查过层级一致性
        和比例缩放，未替换主方案”的结论。
902
903 ## 6. 论文写法建议
904
905 当前主方案仍应冻结为低销量指数平滑--常规 Ridge 混合策略。可以在模型评价中补充：本
        轮尝试了不同低销量阈值、低销量均值兜底和层级校准，虽然近 28 日均值兜底在全样本
        验证中表现更好，但留一窗口检查未达到稳定替换阈值，因此最终预测表继续使用 `
        outputs/final_7day_forecast_hybrid_low_volume.csv`。
906 """
907
908
909 def build_error_report(
910     by_horizon: pd.DataFrame,
```

```
    by_class: pd.DataFrame,
    by_calendar: pd.DataFrame,
    class_summary: dict,
) -> str:
    horizon_cols = ["horizon", "MAE", "RMSE", "WAPE_pct", "actual_sum", "
        prediction_sum", "n"]
    class_cols = [
        "demand_class",
        "is_low_volume",
        "MAE",
        "RMSE",
        "WAPE_pct",
        "actual_sum",
        "prediction_sum",
        "n",
    ]
    calendar_cols = [
        "calendar_segment",
        "definition",
        "MAE",
        "RMSE",
        "WAPE_pct",
        "actual_sum",
        "prediction_sum",
        "date_count",
    ]
    worst_horizon = by_horizon.sort_values("WAPE_pct", ascending=False).iloc
        [0]
    low_wape = class_summary.get("low_volume", {}).get("WAPE_pct", np.nan)
    regular_wape = class_summary.get("regular", {}).get("WAPE_pct", np.nan)

    return f"""# 问题四混合策略误差归因增强报告

执行日期: {date.today().isoformat()}

## 1. 目的

本报告基于当前最终候选模型“低销量指数平滑--常规 Ridge 混合策略”的严格 7 日递推
    验证明细，补充按预测步长、需求类别和日历场景的误差归因。该报告服务于论文解释和
    答辩，不改变最终预测表。
```

```
947
948 ## 2. 按 horizon 的误差
949
950 {df_to_md(by_horizon[horizon_cols], 10)}
951
952 horizon={int(worst_horizon['horizon'])} 的 WAPE 最高，为 {worst_horizon['
    WAPE_pct']:.2f}%。若后续步长误差升高，可解释为递推预测中前序预测值进入后续
    lag/rolling 特征造成误差累积。
953
954 ## 3. 按低销量类别的误差
955
956 {df_to_md(by_class[class_cols], 20)}
957
958 低销量类别聚合 WAPE 约为 {low_wape:.2f}%，常规类别聚合 WAPE 约为 {regular_wape
    :.2f}%。低销量组真实销量规模小，少量绝对误差会被 WAPE 放大，论文应同时引用
    MAE、RMSE 和真实销量合计。
959
960 ## 4. 按日历场景的误差
961
962 {df_to_md(by_calendar[calendar_cols], 20)}
963
964 周末、节假日和活动日样本可能重叠，且验证窗口有限；因此这些结果只能说明模型在不同场
    景上的误差表现，不能作为因果判断。
965
966 ## 5. 输出文件
967
968 - `tables/q4_hybrid_error_by_horizon.csv`
969 - `tables/q4_hybrid_error_by_demand_class.csv`
970 - `tables/q4_hybrid_error_by_calendar_segment.csv`
971 - `figures/q4_hybrid_error_by_horizon.png`
972 """
973
974
975 def build_uncertainty_report(
976     store: pd.DataFrame,
977     product: pd.DataFrame,
978     category: pd.DataFrame,
979 ) -> str:
980     store_cols = [
981         "store_name",
```

```
        "point_forecast_7day_sales",
        "lower_80pct_empirical",
        "median_residual_adjusted_forecast",
        "upper_80pct_empirical",
        "mean_abs_residual",
        "residual_count",
    ]
    category_cols = [
        "category",
        "point_forecast_7day_sales",
        "lower_80pct_empirical",
        "median_residual_adjusted_forecast",
        "upper_80pct_empirical",
        "mean_abs_residual",
        "residual_count",
    ]
    product_cols = [
        "product_name",
        "category",
        "point_forecast_7day_sales",
        "lower_80pct_empirical",
        "upper_80pct_empirical",
        "mean_abs_residual",
    ]

    return f"""# 未来 7 天预测不确定性报告

执行日期: {date.today().isoformat()}

## 1. 区间构造方法

本报告不重新训练模型，而是使用当前混合策略在 4 个严格 7 日递推验证窗口上的 7 日聚合残差构造经验区间。对每个门店、商品或类别，先计算验证窗口中的

`残差 = 真实7日销量 - 预测7日销量`,

再取残差的 10% 和 90% 分位数加到未来 7 天点预测上，形成经验 80% 残差区间。由于验证窗口只有 4 个，该区间不是严格统计置信区间，只用于反映历史递推误差下的预测不确定性。

```

```
## 2. 门店级区间

{df_to_md(store[store_cols], 20)}

完整表见 `tables/final_forecast_interval_by_store.csv`，图见 `figures/
    final_forecast_interval_by_store.png`。

## 3. 类别级区间

{df_to_md(category[category_cols], 20)}

完整表见 `tables/final_forecast_interval_by_category.csv`。

## 4. 商品级区间示例

下表按点预测销量从高到低展示前 15 个商品。完整表见 `tables/
    final_forecast_interval_by_product.csv`。

{df_to_md(product[product_cols], 15)}

## 5. 使用边界

这些区间只来自历史验证残差，不能覆盖未来天气、促销安排、库存缺货或突发大单等未观测
    变化。论文中适合表述为“基于严格递推验证残差的经验不确定性范围”，不应写成保证
    真实销量落入区间的概率承诺。
"""


def write_summary_json(
    metrics: pd.DataFrame,
    store_interval: pd.DataFrame,
    leave_one: pd.DataFrame,
) -> None:
    sp = metrics[metrics["level"] == "store_product"].copy()
    current = sp[sp["model"] == CURRENT_MODEL].iloc[0]
    best = sp.iloc[0]
    selected_mean = float(
        leave_one.loc[leave_one["eval_type"] == "selected_by_other_windows", "
            WAPE_pct"].mean()
    )
```

```
    current_holdout_mean = float(
        leave_one.loc[leave_one["eval_type"] == "current_hybrid", "WAPE_pct"].
            mean()
    )
    holdout_improvement = current_holdout_mean - selected_mean
    summary = {
        "validation_mode": "strict_recursive_7day",
        "current_model": CURRENT_MODEL,
        "current_wape_pct": float(current["WAPE_pct"]),
        "best_candidate_model": str(best["model"]),
        "best_candidate_label": str(best["model_label"]),
        "best_candidate_wape_pct": float(best["WAPE_pct"]),
        "best_minus_current_wape_pct_points": float(
            best["WAPE_pct"] - current["WAPE_pct"]
        ),
        "leave_one_selected_mean_wape_pct": selected_mean,
        "leave_one_current_mean_wape_pct": current_holdout_mean,
        "leave_one_improvement_pct_points": holdout_improvement,
        "replace_current": bool(
            current["WAPE_pct"] - best["WAPE_pct"] >= IMPROVEMENT_THRESHOLD_PCT
            and holdout_improvement >= IMPROVEMENT_THRESHOLD_PCT
            and best["model"] != CURRENT_MODEL
        ),
        "store_interval_rows": int(len(store_interval)),
    }
    (OUTPUTS / "q4_award_iteration_summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
    )


def main() -> None:
    for path in [TABLES, OUTPUTS, FIGURES]:
        path.mkdir(parents=True, exist_ok=True)

    base = load_strategy_base()
    candidates = build_candidate_predictions(base)
    candidates = add_calibrated_candidates(base, candidates)
    save_csv(candidates, "q4_candidate_model_predictions.csv")

    metrics = build_metric_table(candidates)
```

```
    window_metrics = build_window_metric_table(candidates)
    leave_one = build_leave_one_window_selection(candidates)
    save_csv(metrics, "q4_candidate_model_metrics.csv")
    save_csv(window_metrics, "q4_candidate_window_metrics.csv")
    save_csv(leave_one, "q4_candidate_leave_one_window_selection.csv")

    by_horizon, by_class, by_calendar, class_summary = build_error_attribution
        (candidates)
    store_interval, product_interval, category_interval =
        build_forecast_intervals(candidates)
    plot_outputs(metrics, by_horizon, store_interval)

    (OUTPUTS / "q4_candidate_model_review.md").write_text(
        build_candidate_review(metrics, window_metrics, leave_one), encoding="
            utf-8"
    )
    (OUTPUTS / "q4_error_attribution_enhanced_report.md").write_text(
        build_error_report(by_horizon, by_class, by_calendar, class_summary),
            encoding="utf-8"
    )
    (OUTPUTS / "forecast_uncertainty_report.md").write_text(
        build_uncertainty_report(store_interval, product_interval,
            category_interval),
        encoding="utf-8",
    )
    write_summary_json(metrics, store_interval, leave_one)

    sp = metrics[metrics["level"] == "store_product"].copy()
    current = sp[sp["model"] == CURRENT_MODEL].iloc[0]
    best = sp.iloc[0]
    improvement = current["WAPE_pct"] - best["WAPE_pct"]
    selected_mean = float(
        leave_one.loc[leave_one["eval_type"] == "selected_by_other_windows", "
            WAPE_pct"].mean()
    )
    current_holdout_mean = float(
        leave_one.loc[leave_one["eval_type"] == "current_hybrid", "WAPE_pct"].
            mean()
    )
    holdout_improvement = current_holdout_mean - selected_mean
```

```
    replace_note = (
        "全样本达到替换阈值但留一窗口未达阈值，作为探索候选但不替换冻结主方案"
        if improvement >= IMPROVEMENT_THRESHOLD_PCT and holdout_improvement <
            IMPROVEMENT_THRESHOLD_PCT
        else (
            "达到替换阈值，需人工复核候选预测表"
            if improvement >= IMPROVEMENT_THRESHOLD_PCT and best["model"] !=
                CURRENT_MODEL
            else "未达到 0.3 个百分点替换阈值，继续冻结当前混合策略"
        )
    )
    row = (
        f"| {date.today().isoformat()} | 问题四有限优化与不确定性补充 | "
        "processed: modeling_base_table.csv；输入严格递推预测和混合策略预测；未修
            改 data/raw，未覆盖原最终预测表 | "
        "低销量阈值候选、低销量均值兜底、层级比例校准、经验残差区间 | "
        "历史均值、零销量比例、近28日/商品/类别均值、问题一聚合预测校准、严格递推
            残差 | "
        f"当前混合策略 WAPE={current['WAPE_pct']:.2f}%；最佳候选={best['
            model_label']} WAPE={best['WAPE_pct']:.2f}%；"
        f"相对当前全样本改善={improvement:.2f} 个百分点；留一窗口选模改善={
            holdout_improvement:.2f} 个百分点 | "
        f"{replace_note}；新增 horizon、低销量、日历场景误差和预测区间 | "
        "层级校准未使用验证窗口真实销量；预测区间为经验残差范围，不是严格置信区间
            | "
        "将候选复核、不确定性和误差归因写入论文与答辩材料 |"
    )
    prepend_result_log(row)

    print(
        json.dumps(
            {
                "current_wape_pct": float(current["WAPE_pct"]),
                "best_model": str(best["model"]),
                "best_label": str(best["model_label"]),
                "best_wape_pct": float(best["WAPE_pct"]),
                "improvement_pct_points": float(improvement),
                "leave_one_selected_mean_wape_pct": float(selected_mean),
                "leave_one_current_mean_wape_pct": float(current_holdout_mean),
                "leave_one_improvement_pct_points": float(holdout_improvement),
```

```
            "replace_current": bool(
                improvement >= IMPROVEMENT_THRESHOLD_PCT
                and holdout_improvement >= IMPROVEMENT_THRESHOLD_PCT
                and best["model"] != CURRENT_MODEL
            ),
        },
        ensure_ascii=False,
        indent=2,
    )
)


if __name__ == "__main__":
    main()
```

## A.4 AI 工具使用说明

AI 工具主要用于代码辅助、报告润色、审查提示、格式整理和 LaTeX 排版转换。字段解释、负销量口径、模型取舍、问题四验证口径和最终结论均需团队人工确认。AI 不替代团队对数据、模型和提交材料的最终判断。完整记录见 `outputs/ai_usage_record.md`。